

OpenFrame SORTユーティリティリファレンスガイド

OpenFrame/Batch for MVS 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame SORTユーティリティリファレンスガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/Batch for MVS 7.1

ガイドバージョン: v3.1.3

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの`$(INSTALL_PATH)/license/oss_licenses`に記載されている事項を参照してください。

目次

このガイドについて	vii
第1章 SORTユーティリティの紹介	1
1.1. 概要	1
1.2. ユーティリティの機能	1
1.3. SORTの制御文	2
1.3.1. ジョブ制御文	2
1.3.2. データセットの定義	4
1.4. 一般的なコーディング規則	6
第2章 SORTコマンド	9
2.1. 概要	9
2.2. ALTSEQ	10
2.3. DUPKEYS	11
2.4. END	14
2.5. INCLUDE/OMIT	14
2.6. INREC	17
2.7. JOIN	24
2.8. JOINKEYS	25
2.9. MERGE	28
2.10. MODS	31
2.11. OPTION	34
2.12. OUTFIL	38
2.13. OUTREC	60
2.14. RECORD	68
2.15. REFORMAT	70
2.16. SORT	71
2.17. SUM	75
索引	79

図目次

[図 1.1] SORTユーティリティ・コマンドの構文	6
-----------------------------------	---

このガイドについて

対象読者

本書は、リホスト・ソリューションのTmax OpenFrame[®](以下、OpenFrame)/Batchシステムを使用するユーザーを対象としています。

前提知識

本書を理解するには、OpenFrame/Batchの概念を理解する必要があります。本書を読む前に『OpenFrame Batch ガイド』を先にお読みになることをお勧めします。

制限事項

本書では、ユーティリティで現在サポートしていない機能、オプション、パラメータについては別途表記しており、それについての説明は省略しています。

本書の構成

本書は、計2章で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: SORTユーティリティの紹介

SORTユーティリティの機能、制御文およびコーディング規則について説明します。

- 第2章: SORTコマンド

SORTコマンドの使用方法和オペランドについて説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名
Ctrl+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上 (32ビット, 64ビット)
	Linux x86 2.6以上 (32ビット, 64ビット)
	Linux ia64 2.6以上 (32ビット, 64ビット)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 (Fixset07)
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、ティーマックスソフトの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame Batchガイド	OpenFrame/Batchの紹介および機能について説明しています。
OpenFrame ユーティリティ リファレンスガイド	OpenFrameエンジンと一緒に提供される多様なユーティリティ・プログラムについて説明しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

第1章 SORTユーティリティの紹介

本章では、SORTユーティリティの機能、制御文およびコーディング規則について説明します。

1.1. 概要

SORTユーティリティは、IBMメインフレームのDFSORTに対応されるユーティリティ・プログラムであり、入力データセット(SORTIN、SORTINnn)に対してソート、マージ、コピー、フィルタリング、再フォーマット、合計などを実行し、出力データセット(SORTOUT、OUTFILデータセット)を作成します。SORTユーティリティは、サードパーティのソート・ユーティリティであるTmaxSoftのProSortを使用しています。

1.2. ユーティリティの機能

以下は、SORTユーティリティの主な機能です。

区分	説明
ソート	ソートを実行する入力データセットのレコードを、指定されたキーに従って昇順または降順に並べ替えて出力データセットとして出力します。
マージ	SORTINnnに指定された入力データセットのマージを実行するために、指定されたキーを使用してソート済みの2つ以上の入力データセットを、1つのソートされた出力データセットとしてマージして出力します。
コピー	入力データセットのすべてのレコードを出力データセットにコピーします。

入力データセットがTSAMデータセットの場合にSORTユーティリティの主な機能を実行する際、以下のいずれか最初に一致する方法で処理します。

- RECORD TYPE=[F|V]を実行する場合、[F|V]にします。
- SORTOUT DDに指定されたデータセットのRECFMが確定された場合、RECFMにします。
- OUTFIL DDに指定されたデータセットのRECFMが確定された場合、RECFMにします。
- SORTINがTSAM、SORTOUTがTSAMの場合、Vタイプにします。

以下は、SORTユーティリティで実行できるその他の機能です。

- 入力レコードのフィルタリング(条件付き入力フィルタリング)

入力レコードの再フォーマットを行う前に、INCLUDE、OMIT、SKIPREC、STOPAFTなどで条件を満たす入力レコードのみを含めるか除外することができます。

- 入力レコードの再フォーマット

ソート、マージ、コピーを実行する前に、INRECでフィルタリングされたレコードを指定された形式で再フォーマットします。これにより、入力レコードから不要なフィールドを削除し、ソート、マージ、コピーを実行する際の性能を高めることができます。

- フィールド合計

ソート、マージの実行時にSUMを使用してキー値が一致する1つ以上のレコードの値をすべて合計します。EQUALSオプションが設定されていると、フィールドの値をすべて合計した後、同じキー値のレコードのうち、最初のレコードのみをSORTOUTとして出力します。

- 出力レコードの再フォーマット

ソート、マージ、コピーを実行した後、OUTRECを使用して指定された形式でレコードを再フォーマットします。再フォーマットされたレコードがSORTOUTとして出力されるか、OUTFILの入力レコードとなります。

- 出力レコードのコピー

OUTFILを使用してSORTOUTとして書き込まれるレコードを入力レコードとして、複数のOUTFIL DDで出力します。入力レコード・フィルタリングと出力レコードの再フォーマットがこの段階で再適用されることがあります。

1.3. SORTの制御文

本節では、SORTの制御文について説明します。

1.3.1. ジョブ制御文

ジョブ制御文は、JOB文、EXEC文、DD文、/*文などで構成されています。

- JOB文

ジョブの先頭に位置する制御文です。ジョブの開始を示しており、そのジョブの特性を定義します。

以下は、記述例です。

```
//SORT00 JOB CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1)
```

- EXEC文

1つのジョブは複数のステップで構成されており、EXEC文はジョブ・ステップの先頭に位置します。プログラムの名称およびプログラムに渡すパラメータなどを指定します。

以下は、記述例です。

```
//SORT EXEC PGM=sort,REGION=2M
```

以下は、SORTで使用できるパラメータです。

パラメータ	説明
'JPn"string"'	SORTスクリプト内でJPnを文字列に置き換えます。
VLTESTI=n	INCLUDEまたはOMIT演算で指定されたフィールドがレコードに存在しないか、一部のみ存在する場合の動作を定義します。 - VLTESTI=0 : INCLUDEまたはOMIT演算で指定されたすべてのフィールドがレコードの存在する必要があります。そうでない場合は、エラーが発生します。VLTESTIパラメータが指定されていない場合は、VLTESTI=0と見なされます。 - VLTESTI=1 : INCLUDE演算は、指定されたフィールドが入力レコードに存在しないか一部のみ存在すると、そのレコードを出力しません。OMIT演算の場合は、指定されたフィールドが入力レコードに存在しないか一部のみ存在すると、そのレコードを出力します。 - VLTESTI=2 : INCLUDEまたはOMIT演算で指定されたフィールドが入力レコードに存在しないか一部のみ存在すると、その関係式を「偽」と判断します。

● DD文

ソートの実行時に使用されるデータセットの名前や属性などを以下の項目で指定します。

項目	説明
SYSIN	ソートの制御文を入力するためのデータセットです。一般的にDD*でシステムの入カストリームを使用します。
SYSOUT	制御文のリストおよびメッセージを出力するためのデータセットです。一般的にシステムの出カストリームを使用します。
SORTIN	ソート/コピー処理の入力データセットです。
SORTINnn	マージ処理の入力データセットです。データセットの名前は、SORTIN01から順に定義する必要はありませんが、同じ番号を使用することはできません。 また、マージ処理での入力データセットは、以下の条件を満たす必要があります。 - レコード形式(RECFM)は、同じ形式である必要があります。 - レコード形式(RECFM)が固定長(FB)の場合は、同じレコード長(LRECL)である必要があります。 - 入力データセットは、MERGEコマンドのFIELDSオペランドに指定されたキーフィールド条件で事前にソートされている必要があります。
SORTWKnn	ソート処理中に作業領域として使用するデータセットです。

項目	説明
SORTOUT	ソート、マージ、コピーの処理結果を出力するデータセットです。 JCLでSORTOUTを新規作成するとき、レコード形式(RECFM)が指定されていない場合は、SORTINまたはSORTINnnが固定長(FB)であればSORTOUTも固定長、SORTINまたはSORTINnnが可変長(VB)であればSORTOUTも可変長で作成されます。入力が非VSAMのレコードをVSAMレコードとして出力するか、VSAMレコードを非VSAMレコードとして出力することも可能です。
SORTOFxx	OUTFILを使用してレコードを再フォーマットする際、複数の出力データセットで出力します。xxは、1文字または2文字で構成された任意の英数字です。
SYMNAMES	シンボルを置換する場合、参照されるデータセットです。 – レコード形式 シンボル名, フィールドの開始位置, フィールドの長さ, フィールドのフォーマット
JNFnCNTL	JOINを実行する際、subtasknの制御文です。JOINを実行する前に実行するSORT文を指定します。nは、1または2です。

- /*文

SYSINデータセットの終了を指定します。SORT制御文のENDを使用して終了を指定することもできます。

1.3.2. データセットの定義

以下は、データセットの定義に一般的に必要なオペランドです。

データセット	DCB	DISP	DSN	LABEL	SPACE	UNIT	VOL	AMP
SORTIN	1	○	○	2	X	3	3	7
SORTINnn	1	○	○	2	5	3	3	7
SORTOUT	1	○	4	2	5	3	6	7
SORTOFxx	1	○	4	2	5	3	6	7
SORTWKnn	X	4	4	X	5	3	6	X
SYMNAMES	1	○	○	2	X	3	3	X
JNFnCNTL	1	4	4	2	5	3	3	7

- ○：必ず指定します。
- X：指定しなくても構いません。
- 1：場合によって指定が必要です。(コマンドの内容に応じて)
- 2：ディスク上では不要です。
- 3：現在のジョブ・ステップの前に作成およびカタログングされている場合は不要ですが、そうでない場合は指定する必要があります。
- 4：一時データセットでない場合は指定します。
- 5：直接アクセス記憶装置上に作成する場合は指定します。
- 6：一時データセットの場合、現在のステップの前に作成およびカタログングされていれば、同じジョブ内の他のステップから渡された場合を除いて指定します。
- 7：VSAMデータセットの場合は、付加情報が必要な際に指定することができ、それ以外のデータセットの場合は不要です。

DCBオペランドのサブパラメータ

入出力データセットがVSAMデータセットでない場合、レコード形式(RECFM)、レコード長(LRECL)、ブロック・サイズ(BLKSIZE)の情報を指定する必要があります。

項目	説明
レコード形式(RECFM)	データセットに記録されているレコード形式を指定します。
レコード長(LRECL)	データセットに記録されているレコードの論理的な最大長をバイトで指定します。
ブロック・サイズ(BLKSIZE)	データセットに記録されているレコードの物理的な最大長をバイトで指定します。

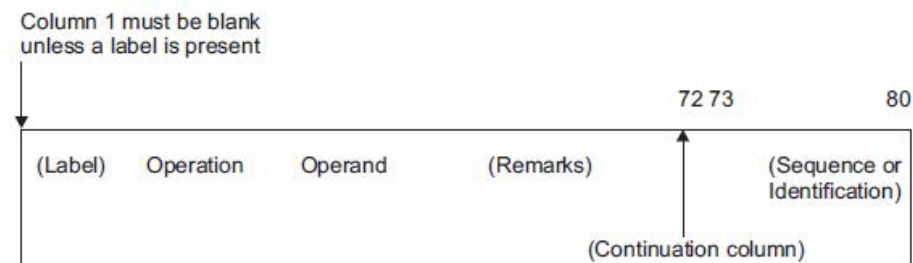
上記情報の指定は、以下の順序に従って決定されます。

1. RECFM、LRECL、BLKSIZEがDD文のDCBオペランドで指定されている場合はその情報を使用し、その他の設定情報は無視します。
2. 入力データセットの場合、DCBオペランドが指定されていないとエラーが発生します。ただし、すでにデータセットが存在している場合はその情報を使用します。出力データセットの場合、DCBオペランドの指定がなければ以下の順序で指定されます。
 - a. OUTRECで指定された情報を上書きして使用します。
 - b. RECORDで指定された情報を上書きして使用します。
 - c. INRECで指定された情報を上書きして使用します。
 - d. 入力データセットで指定された情報を上書きして使用します。

1.4. 一般的なコーディング規則

以下は、SORTユーティリティのコマンド文を作成するための構文です。

[図 1.1] SORTユーティリティ・コマンドの構文



区分	説明
ラベル・フィールド(Label Field)	ラベル・フィールドは行の1列目から始まります。空白やアスタリスク(*)は1列目に使用できません。（ラベル・フィールドは省略可能） ラベルは1～70列まで使用可能であり、空白が現れるとラベルの終わりを示します。
オペレーション・フィールド(Operation Field)	SORT、MERGEなど、プログラムが実行するコマンドを指定します。1行目の2～71列に作成できます。
オペランド・フィールド(Operand Field)	オペランド・フィールドはコンマで続けることができ、オペレーション・フィールドの後ろに少なくとも1つ以上の空白文字を入れて指定します。 オペランド・フィールド内では、空白は使用できません。
リマーク・フィールド(Remark Field)	コメント文のようにどんな内容でも作成することができます。プログラムの動作中には無視されます。 オペランド・フィールドの後ろに1つ以上の空白文字を入れて作成します。
継続列(Continuation Column)(72)	72列目は継続列です。この列に空白以外の文字が位置すると、現在の構文が次の行に続くことを意味します。ただし、オペランド・フィールドがコンマで終わった場合は、次の行をオペランドの続きと判断します。
73～80列	このフィールドは自由に使用できます。プログラムでは読み込みません。
継続行(Continuation Lines)	継続行は、先行する行の論理的な拡張です。これはオペランドの継続であることもあり、コメント文の継続である場合もあります。 – オペランドの継続：2～71列内でオペランドがコンマで終わった場合、次の行をオペランドの継続と見なします。また、71列目ま

区分	説明
	でオペランドが続き、72列目に空白でない文字が位置する場合も次の行に続きます。 – コメント文の継続：2～71列内でオペランドが終わり、最後の文字がコンマでない場合、72列目に空白でない文字があれば、次の行はコメント文の継続と見なし、プログラムでは無視します。

注

1列目がアスタリスク(*)で始めると、その行はコメント文と見なされます。コマンド文内に空白行が存在する場合、プログラムはこれを無視します。したがって、空白行は実行の続きに影響を与えません。

第2章 SORTコマンド

本章では、SORTコマンドの使用方法和オペランドについて説明します。

2.1. 概要

以下は、OpenFrameが提供するSORTコマンド(operator)の一覧です。各コマンドの使用方法和オペランドについては、各節で詳しく説明します。

コマンド	説明
ALTSEQ	ALTSEQの変換テーブルを使用してソート、マージ、コピーなどを実行する際、特定の文字を指定された順に変更します。
DUPKEYS	ソート、マージを実行する際、同じキー値を持つレコードに対して指定されたフィールドの値を演算し、1つのレコードとして表示します。
END	SORT文の終わりを示します。
INCLUDE/OMIT	INCLUDE文は、設定された条件を満たすレコードのみを対象にソート/マージを実行します。OMIT文は、設定された条件を満たすレコードを除いたレコードのみを対象にソート/マージを実行します。
INREC	ソート、マージ、コピーを実行する前に、指定された形式に従って入力レコードを再フォーマットします。
JOINKEYS, JOIN, REFORMAT	SORTJNF1とSORTJNF2に記述した入力データセットを出力データセットに結合する際に使用します。
MERGE	SORTINnnに記述したnn個の入力データセットを出力データセットにマージする際に使用します。
MODS	MODS制御文を使用したスクリプトを使用してユーザー関数を登録できます。
OPTION	オプションを指定します。
OUTFIL	SORTOUTをOUTFIL DDに指定した複数のデータセットに保存します。
OUTREC	ソート、マージ、コピーを実行した後、その結果をSORTOUTやOUTFILで指定したデータセットに出力する前に出力レコードを再フォーマットします。
RECORD	レコードの形式および長さを指定します。
SORT	与えられたキーに従って入力データセットのレコードを昇順または降順にソートし、出力データセットに出力します。

コマンド	説明
SUM	ソート、マージを実行する際、同じキー値を持つレコードに対して指定されたフィールドの値をすべて合計し、1つのレコードとして表示します。

2.2. ALTSEQ

ALTSEQの変換テーブルを使用してソート、マージ、コピーなどを実行する際、特定の文字を指定された順に変更します。これにより、キーの順序を変更するか、異なるフィルタリング条件を適用することができます。OpenFrameではASCII順に従います。

参考

順序の変更は、ソートまたはマージの実行時にのみ適用され、レコードの出力内容は変更されません。

構文

```
ALTSEQ CODE=(xxyy,xxyy,...,xxyy)
```

オペランド

以下は、オペランドについての説明です。

• CODE

- 変更する既存の文字と変更される新しい文字の順序を指定します。xxyyの指定に従って変更する文字の順序変更処理を行います。
- 以下は、項目についての説明です。

項目	説明
xx	順序を変更する文字を16進数の2桁で指定します。
yy	新しく変更される順序を指定します。

- 以下は、ソートの実行時、1(ASCII番号：31)の順序をa(ASCII番号：61)に変更し、昇順にソートする例です。

```
SORT FILEDS=(1,8,AQ,A)
ALTSEQ CODE=(3161)
```

2.3. DUPKEYS

ソート、マージを実行する際、同じキー値を持つレコードに対して指定されたフィールドの値を演算し、1つのレコードとして表示します。表示されたレコードを除いて、残りのレコードはすべて削除されます。また、キー値が同じレコードのフィールドの演算を行わずに、レコードを1つだけ残し、その他のレコードはすべて削除する機能も提供します。

構文

```
DUPKEYS function[,function]...[,FORMAT=f] |  
  FIELDS=NONE|ALLDUPS|FIRSTDUP[,NODUPS]|LASTDUP[,NODUPS]|NODUPS|[,XDUP]  
where function is  
AVG|MAX|MIN|SUM=(p1,l1[,f1][,p2,l2[,f2]]...)
```

オペランド

以下は、オペランドについての説明です。

- FORMAT=f
 - 関数で使用するフィールドの形式がすべて同じである場合は、各フィールド内で形式を指定せずに、CONDオペランドの後ろにFORMAT=fを使用して一度に指定することができます。
 - 以下は、10から2バイト、12から4バイト、16から8バイトのFI形式のレコードを合計する例です。

```
DUPKEYS SUM=(10,2,12,4,16,8),FORMAT=FI
```

参考

OpenFrameでは制御フィールドの形式がすべて同じである場合にのみFORMATオペランドを指定することができます。

- FIELDS=NONE
 - レコードを演算せずに、同じキー値を持つレコードを1つだけ残して残りはすべて削除する場合に使用します。
 - 以下は、同じキー値を持つレコードを1つだけ残してすべて削除する例です。

```
DUPKEYS FIELDS=NONE
```

- ALLDUPS
 - レコードを演算せずに、同じキー値を持つレコードだけ残して残りはすべて削除する場合に使用します。

- 以下は、同じキー値を持つレコードだけ残してすべて削除する例です。

```
DUPKEYS ALLDUPS
```

- FIRSTDUP[,NODUPS]

- レコードを演算せずに、同じキー値を持つレコードのうち最初のレコードだけ残してすべて削除する場合に使用します。NODUPSオペランドを追加する場合は、同じキー値を持たないレコードも残します。
- 以下は、同じキー値を持つレコードのうち最初のレコードと同じキー値を持たないレコードを残してすべて削除する例です。

```
DUPKEYS FIRSTDUP, NODUPS
```

- LASTDUP[,NODUPS]

- レコードを演算せずに、同じキー値を持つレコードのうち最後のレコードだけ残してすべて削除する場合に使用します。NODUPSオペランドを追加する場合は、同じキー値を持たないレコードも残します。
- 以下は、同じキー値を持つレコードのうち最後のレコードだけ残してすべて削除する例です。

```
DUPKEYS LASTDUP
```

- NODUPS

- レコードを演算せずに、同じキー値を持つレコードをすべて削除し、同じキー値を持たないレコードだけを残す場合に使用します。
- 以下は、同じキー値を持つレコードをすべて削除し、同じキー値を持たないレコードだけを残す例です。

```
DUPKEYS NODUPS
```

- XDUP

- DUPKEYSコマンドの処理時に削除されるレコードをDD名がSORTXDUPであるデータセットに保存する場合に使用します。
- 以下は、同じキー値を持つレコードをDD名がSORTXDUPであるデータセットに保存する例です。

```
DUPKEYS NODUPS, XDUP
```

- AVG|MAX|MIN|SUM

- 入力レコードを演算するためのキー・フィールドを指定します。

項目	説明
AVG	平均演算

項目	説明
MAX	最大値
MIN	最小値
SUM	合計

- キー・フィールドの開始位置と長さ、形式を指定できます。
- 以下は、設定項目についての説明です。

項目	説明
p	演算を実行するフィールドの開始位置を指定します。詳細については、 「2.16. SORT」 のFIELDSオペランドのp項目を参照します。
m	演算するフィールドの長さをバイト単位で指定します。
f	演算フィールドの形式を指定します。
s	制御フィールドのソート順を指定します。 - A (Ascending order) : 昇順 - D (Descending order) : 降順 - E : ユーザー定義の順序（OpenFrameではサポートしていません）

- 以下は、ZD形式のレコードに対して開始位置10から10バイトを合計し、開始位置5から4バイトを平均演算する例です。

```
DUPKEYS SUM=(10,10,ZD),AVG=(5,4,ZD)
```

注意事項

以下は、DUPKEYS文を作成する際の注意事項です。

- ソート、マージでNOEQUALSが指定されている場合は、演算および削除されるレコードはランダムに決定されます。
 - ソートで、EQUALSが指定されている場合は、先に入力されたレコードに残りのレコードを演算します。
 - マージで、EQUALSが指定されている場合は、SORTINnn番号が一番小さいレコードに合算されます。SORTINnn番号が同じである場合は、先に入力されたレコードに合算されます。
- DUPKEYSの実行時に、指定したレコード・フィールドの内容だけ演算され、他のフィールドの内容は演算されません。

- DUPKEYSの実行時に、オーバーフローが発生した場合は、演算を行いません。また、演算が行われなかったレコードは削除されません。

2.4. END

SORT文の終わりを示します。END文以降はコマンドを処理しません。END文の付加的なオペランドはありません。

参考

END文は、直後に「/*」または、それ以外のジョブ制御文がある場合は省略が可能です。

構文

```
END
```

2.5. INCLUDE/OMIT

入力レコードに対してソート、マージを実行する前に、任意で条件を設定する機能です。

INCLUDE文は設定された条件を満たすレコードのみを対象にソート、マージを実行し、OMIT文は設定された条件を満たすレコードを除いた残りのレコードのみを対象にソート、マージを行います。ソート、マージを実行する前に入力relational_conditionレコードのサイズを小さくして、作業効率を高めることができます。

参考

INCLUDEコマンドとOMITコマンドを一緒に指定することはできません。

構文

```
{INCLUDE | OMIT} COND=(logical_expression) [,FORMAT=f]
logical_expression=relational_condition1,[AND | OR],[,relational_condition2[,...]]
relational_condition=([(p1,m1,f1)],[EQ|NE|GT|GE|LT|LE],[,(p2,m2,f2)|NUM|constant])
```

オペランド

以下は、オペランドについての説明です。

- COND
 - ソート、マージを実行するレコードの条件を指定します。

- ソート、マージ、コピーを実行する入力レコードの条件を指定する部分です。論理式(logical_expression)の各比較条件(relational_condition)を演算した値が真の場合、条件を満たします。論理演算子としてANDとORを使用して、2つ以上の比較条件をこの論理演算子で結合します。ANDは「&」に、ORは「|」に置き換えて使用できます。
- 以下は、論理式を使用したCONDの例です。CH形式の5から8バイト分のレコードが、CH形式の13から8バイト分のレコードより大きい場合、あるいはBI形式の105から4バイト分のレコードが、1000より小さいか等しい場合、そのレコードを含めます。

```
INCLUDE COND=(5,8,CH,GT,13,8,CH,OR,105,4,BI,LE,1000)
```

- FORMAT=f

- 比較条件(relational_condition)のフィールド形式がすべて同じである場合は、各フィールド内で形式を指定せずに、CONDオペランドの後ろにFORMAT=fを使用して一度に指定することができます。
- 以下は、COND条件のすべてのフィールドの形式が同じである場合に、FORMAT=fを使用する例です。
 - 条件1: CH形式のレコード・フィールドの5から1バイトと8から1バイトが同じである。
 - 条件2: CH形式のレコード・フィールドの10から1バイトがAまたはBであったり、30から1バイトがCでもDでもない。

条件1と条件2の両方を満たすレコードを除外します。

```
OMIT  
COND=((5,1,EQ,8,1),&,(10,1,EQ,C'A',|,10,1,EQ,C'B'),|,(30,1,NE,C'C',&30,1,NE,C'D'))),FORMAT=CH
```

参考

OpenFrameでは、FORMAT=fをCONDオペランドの後ろにのみ指定できます。また、制御フィールドの形式がすべて同じである場合にのみ指定できます。

- 比較条件(relational_condition)

入力レコードの比較条件を指定します。p1、m1、f1はフィールド1の情報であり、p2、m2、f2はフィールド2の情報です。

- p

フィールド1とフィールド2の開始点を指定します。詳細については、[「2.16.SORT」](#)のFIELDSオペランドのp項目を参照してください。

- m1, m2

フィールド1とフィールド2の開始点からの相対的な長さをバイトで指定します。

- f1, f2

フィールド1とフィールド2の形式を指定します。以下は、指定可能な形式および長さについての説明です。

形式	説明	長さ	符号
CH	文字(ASCII)	1～32760バイト	なし
AC	文字(ASCII)	1～32760バイト	なし
ZD (Zoned Decimal)	ゾーン10進数	1～18バイト	あり
PD (Packed Decimal)	パック10進数	1～9バイト	あり
FI (Fixed-point)	固定小数点2進数	1～8バイト	あり
BI (Binary Integer)	2進数	1～8バイト	なし
CSFまたはFS	追加の前置浮動付数字	1～17バイト	あり
UFF (Unsigned Free Format)	符号がない自由形式の数字	1～17バイト	なし
SFF (Signed Free Format)	符号がある自由形式の数字	1～17バイト	あり
ASL	前置分離符号付数字、ASCII	1～18バイト	あり
AST	後置分離符号付数字、ASCII	1～18バイト	あり
AQ	文字(ALTSEQで定義した順序)	1～32760バイト	なし

– EQ|NE|GT|GE|LT|LE

各比較フィールドは比較演算子により結合され、比較演算子が指定する条件を満たす場合、真となります。以下は、比較演算子の種類です。

比較演算子	説明
EQ	フィールド1 = フィールド2
NE	フィールド1 ≠ フィールド2
GT	フィールド1 > フィールド2
GE	フィールド1 ≥ フィールド2
LT	フィールド1 < フィールド2
LE	フィールド1 ≤ フィールド2

– NUM

フィールドが数字であるかどうかを判断する関係式です。

```
x1y1...xmym
```

フィールドを16進数で表現した結果が上記のとおりであれば、演算子がEQの場合の関係式の結果は以下のとおりです。

- f1がCSF(またはFS)であるか、FORMAT=CSF(またはFS)に指定された場合： $x_1=\dots=x_m=3$ で、 $0\leq y_1,\dots,y_m\leq 9$ であれば、関係式は真になり、それ以外の場合は関係式が偽になります。
 - fがZDであるか、FORMAT=ZDに指定された場合： $x_1=\dots=x_{m-1}=3$ で、 $x_m\in\{3,4,5,7\}$ で、 $0\leq y_1,\dots,y_m\leq 9$ であれば、関係式は真になり、それ以外の場合は関係式が偽になります。
 - fがPDであるか、FORMAT=PDに指定された場合： $0\leq x_1,\dots,x_m\leq 9$ で、 $0\leq y_1,\dots,y_{m-1}\leq 9$ で、 $y_m\in\{C,D,F\}$ であれば、関係式は真になり、それ以外の場合は関係式が偽になります。
- 演算子がNEの場合の関係式の結果は、EQ演算子を使用した場合と反対になります。

– 定数(constant)

10進数(n, +n, -n)、文字列(C'xx...x')、16進文字列((X'yy...y'))が指定可能です。また、現在の日付 (DATE1)、未来の日付 (DATE1+n)も文字列で使用可能です。

参考

定数で符号を付ける場合は、必ず数字の前に付けます。また、文字列はアポストロフィ(')で囲み、文字列の中にアポストロフィ(')を含める場合は、アポストロフィ(')を2つ連続して使用します。

2.6. INREC

ソート、マージ、コピーを実行する前に、入力レコードを再編集します。

構文

```
INREC PARSE=(parse_item[,parse_item[,...]]) |  
      FIELDS=(reformat_command[,reformat_command[,...]]) |  
      BUILD=(reformat_command[,reformat_command[,...]])  
      OVERLAY=(reformat_command[,reformat_command[,...]])  
      FINDREP={IN=incon,OUT=outcon | IN=(incon1,incon2[,...]),OUT=outcon |  
              INOUT=(incon1,outcon1[,incon2,outcon2[,...]])}  
      [,STARTPOS=p |ENDPOS=q]  
      IFTHEN=WHEN=INIT,{BUILD | OVERLAY | FINDREP} |  
              WHEN=GROUP,BEGIN=(logexp),END=(logexp),RECORDS=n,PUSH=(items) |  
  
              WHEN=(logexp),{BUILD | OVERLAY | FINDREP} |  
              WHEN=NONE,{BUILD | OVERLAY | FINDREP}
```

[parse_item]

```
parse_item=%nn= { FIXLEN=m | ABSPOS=p | ADDPOS=x | SUBPOS=y |  
                  STARTAFT=string | STARTAFT=BLANKS |
```

```
STARTAT=string | STARTAT=BLANKS | STARTAT=NONBLANK |
ENDBEFR=string | ENDBEFR=BLANKS |
ENDAT=string | ENDAT=BLANKS |
PAIR=APOST | PAIR=QUOTE }
```

[reformat_command]

```
reformat_command=[c:] | s | %nn | p,m | p,m,TRAN=LTOU|UTOL | p,m,TRAN=ALTSEQ |
p,m,Y2x | p,m,f,edit | p,m,f,to | arexp,edit | p,m,CHANGE | p,m,a |

DATEn | DATE | &DATE | DATE=(abcd) | &DATE=(abcd) | TIME | &TIME |
p,m,f,dateop | p,m,Yxx,todate | TIME=(abc) | &TIME=(abc) | seqnum |
p,m,justify | %nn,justify | p,m,squeeze | %nn,squeeze
```

オペランド

以下は、オペランドについての説明です。

● PARSE

- 可変長入力レコードから解析済みの固定長フィールドを抽出します。解析済みフィールドは、BUILDまたはOVERLAYの(p,m)を使用できる場所で使用できます。
- 以下は、オプションについての説明です。

オプション	説明
%nn	可変長入力レコードを固定長出力レコードに変換します。 解析済みフィールドは、BUILDまたはOVERLAYの(p,m)を使用できる場所で使用できます。
FIXLEN=m	解析済みフィールドの長さを指定します。この属性は、すべての解析済みフィールドに存在する必要があります。
ABSPOS=p	入力レコードで解析開始位置のpを指定します。
ADDPOS=x	入力レコードで解析開始位置がxだけ増加します。
SUBPOS=y	入力レコードで解析開始位置がyだけ減少します。
STARTAFT	- STARTAFT=string : 入力レコードの最初の文字列の後から解析済みフィールドのレコードを開始します。 - STARTAFT=BLANKS : 入力レコードの最初のブランク・グループの後から解析済みフィールドのレコードを開始します。
STARTAT	- STARTAT=string : 入力レコードの最初の文字列から解析済みフィールドのレコードを開始します。 - STARTAT=BLANKS : 入力レコードの最初のブランクから解析済みフィールドのレコードを開始します。

オプション	説明
	– STARTAT=NONBLANK : 入力レコードの最初の実空白から解析済みフィールドのレコードを開始します。
ENDBEFR	– ENDBEFR=string : 入力レコードの最初の文字列の前のバイトで解析済みフィールドのレコードを終了します。 – ENDBEFR=BLANKS : 入力レコードの最初の実空白の前のバイトで解析済みフィールドのレコードを終了します。
ENDAT	– ENDAT=string : 入力レコードの最初の文字列の最後のバイトで解析済みフィールドのレコードを終了します。 – ENDBEFR=BLANKS : 入力レコードの最初の実空白・グループの終わりで解析済みフィールドのレコードを終了します。
PAIR	– PAIR=APOST : アポストロフィ(')で囲まれた領域は、文字列または実空白として扱われません。 – PAIR=QUOTE : 引用符(")で囲まれた領域は、文字列または実空白として扱われません。

● FIELDS | BUILD

- 入力レコードで選択するキー・フィールドを指定します。キー・フィールドの開始位置と長さを指定できます。フィールドが複数指定されている場合、後ろに指定されたフィールドは、すぐ前のフィールドの後ろに付いて出力レコードとして再フォーマットされます。可変長レコード(VB)の場合、再フォーマット結果レコードの長さが変更されると、それに従ってレコード記述語(RDW)の内容も変更されます。
- FIELDSの代わりにBUILDを使用することができます。
- 以下はオプションについての説明です。

オプション	説明
c:	c:オプションの直後に位置するフィールドの再フォーマットされた入力レコードでの開始位置を指定します。FBの場合はレコードの先頭を、VBの場合はRDWの先頭をそれぞれ1とします。 c:で指定する位置は、以前の選択フィールドや挿入された空白と異なる必要があります。このオプションによって定義されていない区間が生じた場合、その区間は空白で埋められます。
s	分離レコードとして文字列や空白を指定します。 – nX : n個の空白文字を挿入します。 – nC'xx...x' : 文字列を挿入します。nは文字列の繰り返し回数です。

オプション	説明
	– nX'yy...yy' : 16進数の文字列を挿入します。nは文字列の繰り返し回数です。
DATEn	現在の日付を出力します。OpenFrameではDATE1のみサポートしており、「C'yyyymmdd'」形式で表示されます。
TIMEEn	現在の時間を出力します。 nは1~3まで指定でき、以下の形式で表示されます。 – 1 : C'hhmmss' – 2 : C'hhmm' – 3 : C'hh'
DATE=(abcd) または &DATE=(abcd)	現在の日付を「C'adbdc'」形式で出力します。dは区切り文字です。 a,b,c∈{M,D,Y,4} であり、それぞれの意味は以下のとおりです。 – M : 月(01~12)を2桁の数字で表示します。 – D : 日(01~31)を2桁の数字で表示します。 – Y : 年の下2桁を表示します。 – 4 : 年を4桁の数字で表示します。
DATE または &DATE	現在の日付を「C'mm/dd/yy'」形式で出力します。 – mm : 月(01-12) – dd : 日(01-31) – yy : 年の下2桁
DATENS=(abc) または &DATENS=(abc)	現在の日付を「C'abc'」形式で出力します。 a,b,c∈{M,D,Y,4} であり、それぞれの意味は以下のとおりです。 – M : 月(01~12)を2桁の数字で表示します。 – D : 日(01~31)を2桁の数字で表示します。 – Y : 年の下2桁を表示します。 – 4 : 年を4桁の数字で表示します。
TIME または &TIME	現在の時間を「C'hh:mm:ss'」形式で出力します。 – hh : 時間(00-23) – mm : 分(00-59) – ss : 秒(00-59)

オプション	説明
TIME=(abc) または &TIME=(abc)	時間を24時間制で表示するか、あるいは12時間制で表示するかを指定します。 – ab : 24または12を指定できます。24は24時間制、12は12時間制を示します。 – c : 区切り文字であり、「C'hhcmmcss'」形式で挿入されます。
p,m	指定された入力レコードのフィールドを出力フィールドに出力します。 – p : 制御フィールドの開始位置を指定します。詳細については、「2.12. OUTFIL」のOUTRECオペランドのp項目を参照してください。 – m : 選択フィールドの長さをバイト数で指定します。
%nn	%nnラベルの解析済みレコード・フィールドを出力フィールドに出力します。
p,m,TRAN=LTOU UTOL	– p, m : 前述の項目説明と同様です。 – TRAN=LTOU : フィールドの小文字を大文字に変換して出力します。 – TRAN=UTOL : フィールドの大文字を小文字に変換して出力します。
p,m,TRAN=ALTSEQ	– p, m : 前述の項目説明と同様です。 – TRAN=ALTSEQ : フィールドのレコードをALTSEQ変換表に従って変更します。
p,m,Y2x	– p : 前述の項目説明と同様です。 – m : 年を示す出力フィールドの長さを指定します。 – Y2x : OpenFrameではY2Cのみサポートしており、「C'yyyy'」形式で出力されます。
p,m,f,edit	指定のフィールドを定義された形式に変換してOUTFILレコードとして出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,f,to	入力フィールドの形式を変更して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
arexp,edit	算術式を編集形式に変換して入力レコードを再編集します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。

オプション	説明
p,m,CHANGE	入力フィールドの指定された文字列を他の文字列に置換します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,a	指定の入力フィールドを出力フィールドに出力します。アライメントが合わない場合、フィールドの前にヌル文字(0x0)を埋めて位置合わせをします。 aは、H、F、Dのいずれかを指定します。 – H : ハーフワード(2バイト) – F : フルワード(4バイト) – D : ダブルワード(8バイト)
p,m,f,dateop	指定された入力フィールドに、以下のように日付操作を行います。 – ADDDAYS : 指定された入力フィールドに指定の日数を加算します。 – ADDMONS : 指定された入力フィールドに指定の月数を加算します。 – ADDYEARS : 指定された入力フィールドに指定の年数を加算します。 – SUBDAYS : 指定された入力フィールドに指定の日数を減算します。 – SUBMONS : 指定された入力フィールドに指定の月数を減算します。 – SUBYEARS : 指定された入力フィールドに指定の年数を減算します。
p,m,Yxx,todate	指定された入力フィールドをtodateに指定したオプションで変換します。 todateに指定できる値は以下のとおりです。 – DT=(abcd) : • a、b、c : 年(最後の2桁を示すY、4桁の年を示す4)をどのように表示するかと、月、日、年を表示する順序を示します。 • d : 年、月、日を区切るために使用します。 – TOJUL=[Yaa Yaa(s)] : 入力フィールドをユリウス日付形式で出力します。 • Yaa : ユリウス日付の形式を指定します。 • Y2T : C'yysddd' C'yyddd' • Y2W : C'dddsyy' C'dddy'

オプション	説明
	• Y2U : X'yydddf' • Y2X : X'dddyf' • Y4T : C'ccyysddd' C'ccyyddd' • Y4W : C'dddscyy' C'dddccyy' • Y4U : X'ccyydddf' • Y4X : X'dddccyyf' • s : 区切り文字を指定します。 – TOGREG=[Yaa Yaa(s)] : 入力フィールドをグレゴリオ日付形式で出力します。 • Yaa : グレゴリオ日付の形式を指定します。 • Y2T : C'yysmmsdd' C'yymmdd' • Y2W : C'mmsddsyy' C'mmddy' • Y2V : X'0yyymmddf' • Y2Y : X'0mmddyf' • Y4T : C'ccyysmmsdd' C'ccyymmdd' • Y4W : C'mmsddscyy' C'mmddccyy' • Y4V : X'0ccyymmddf' • Y4Y : X'0mmddccyyf' • s : 区切り文字を指定します。
seqnum	出力レコードのシーケンス番号を出力します。シーケンス番号は、入力レコードの後に追加されます。 以下は、OpenFrameでサポートされる形式です。 <pre>SEQNUM,n,fs[,START=j][,INCR=i][,RESTART=(p,m)]</pre> – SEQNUM : シーケンス番号を出力するコマンドを指定します。 – n : シーケンス番号の桁数を指定します。必須オプションです。 – fs : シーケンス番号の文字列形式を指定します。必須オプションです。 – START=j : シーケンス番号の開始値をjに設定します。このオプションは省略可能であり、省略した場合は、START=1と見なされます。

オプション	説明
	– INCR=i:シーケンス番号の増分値をiに設定します。このオプションは省略可能であり、省略した場合は、INCR=1と見なされます。 – RESTART=(p,m):このオプションは省略できます。指定されたフィールドが前のレコードのフィールドと一致しない場合、SEQNUMはjに初期化されます。
p,m,justify	指定されたフィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
%nn,justify	%nnラベルの解析済みレコード・フィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,squeeze	指定されたフィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
%nn,squeeze	%nnラベルの解析済みレコード・フィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。

- OVERLAY

- 出力レコードに上書きする項目を指定します。上書きされていない列はそのまま出力します。

- FINDREP

- 入力レコードで指定された変数を検索して他の変数に置換します。
- 変数の置換の詳細については、[「2.12. OUTFIL」](#)のFINDREPオペランドを参照してください。

- IFTHEN

- 指定された条件に従って、各レコードを異なる方式で再フォーマットできます。
- 条件設定の詳細については、[「2.12. OUTFIL」](#)のIFTHENオペランドを参照してください。

2.7. JOIN

結合を実行する際に、結合条件と一致するレコードを出力するかどうか、また残りのレコードを出力するかどうかを指定します。JOIN文を指定しない場合、両方のデータセット（SORTJNF1, SORTJNF2）で結合条件と一致するレコードのみを出力します。（データベース管理システムの内部結合と似た機能です。）

構文

```
JOIN UNPAIRED, [F1|F2|ONLY][, [F1|F2|ONLY][, ...]]
```

オペランド

以下の6つのオペランドの組み合わせをサポートします。

- UNPAIRED,F1,F2またはUNPAIRED
 - 両方のデータセットで結合条件と一致するレコードだけでなく、左のデータセット(SORTJNF1)の残りのレコードと右のデータセット(SORTJNF2)の残りのレコードも出力します。（データベース管理システムの完全外部結合と似た機能です。）
- UNPAIRED,F1
 - 両方のデータセットで結合条件と一致するレコードだけでなく、左のデータセットの残りのレコードも出力します。（データベース管理システムの左外部結合と似た機能です。）
- UNPAIRED,F2
 - 両方のデータセットで結合条件と一致するレコードだけでなく、右のデータセットの残りのレコードも出力します。（データベース管理システムの右外部結合と似た機能です。）
- UNPAIRED,F1,F2,ONLYまたはUNPAIRED,ONLY
 - 両方のデータセットで結合条件と一致するレコードは出力せずに、左右データセットの残りのレコードのみを出力します。
- UNPAIRED,F1,ONLY
 - 両方のデータセットで結合条件と一致するレコードは出力せずに、左のデータセットの残りのレコードのみを出力します。
- UNPAIRED,F2,ONLY
 - 両方のデータセットで結合条件と一致するレコードは出力せずに、右のデータセットの残りのレコードのみを出力します。

以下は、両方のデータセットで結合条件と一致するレコードは出力せずに、左のデータセットの残りのレコードのみを出力する例です。

```
JOINKEYS FILE=F1,FIELDS=(1,8,A)
JOINKEYS FILE=F2,FIELDS=(1,8,A)
JOIN UNPAIRED,F1,ONLY
REFORMAT FIELDS=(F1:1,80)
```

2.8. JOINKEYS

結合を実行する際に、結合対象データセットと結合フィールド、ソートの有無などを指定します。

入力データセットのRECFMがVタイプの場合は、レコードの処理時にRDWを考慮する必要があります。

構文

```
JOINKEYS {{FILE|FILES}={F1|F2}|F1=ddname|F2=ddname}, FIELDS=(p,m,s[,p,m,s[...]])  
[,SORTED|INCLUDE=(logical_expression)|OMIT=(logical_expression)]+
```

オペランド

以下は、オペランドについての説明です。

- FILES

- FILES=F1と記述すると、結合する際にSORTJNF1を左のデータセットとします。FILES=F2と記述すると、結合する際にSORTJNF2を右のデータセットとします。（別名: FILE）
- 以下は、SORTJNF1とSORTJNF2を結合する例です。

```
JOINKEYS FILE=F1, FIELDS=(1,7,A)  
JOINKEYS FILE=F2, FIELDS=(1,7,A)
```

- F1

- ddnameのデータセットを左のデータセットとします。
- 以下は、DD1とSORTJNF2を結合する例です。

```
JOINKEYS F1=DD1, FIELDS=(1,7,A)  
JOINKEYS FILE=F2, FIELDS=(1,7,A)
```

- F2

- ddnameのデータセットを右のデータセットとします。
- 以下は、SORTJNF1とDD2を結合する例です。

```
JOINKEYS FILE=F1, FIELDS=(1,7,A)  
JOINKEYS F2=DD2, FIELDS=(1,7,A)
```

- FIELDS

- 入力レコードを結合する基準となる結合フィールドを指定します。フィールドの開始位置と長さ、昇順または降順を指定できます。
- 以下は、オプションについての説明です。

オプション	説明
p	制御フィールドの開始位置を指定します。制御フィールドは場合によって開始点が異なります。 – 入力レコードが非VSAM固定長(FB)の場合、入力レコードの1番目のバイトが1となります。 – 入力レコードが非VSAM可変長(VB)の場合、レコードの前に4バイトのレコード記述語(RDW)が追加されるので、RDWの開始点が1となり、実際のレコードは5から始まります。 – 入力レコードがVSAMデータセットであり、OpenFrame SORTでこれを可変長レコードとして処理する場合、RDWをレコードの前に追加し、その開始点を1とします。したがって、結合を実行する際に実際のデータは5から始まります。
m	制御フィールドの長さを指定します。
s	制御フィールドのソート基準を指定します。 – A (Ascending order) : 昇順 – D (Descending order) : 降順

- 以下は、SORTJNF1の1番目のバイトから7バイトまで、SORTJNF2の1番目のバイトから7バイトまでの内容を基準に結合する例です。

```
JOINKEYS FILE=F1, FIELDS=(1, 7, A)
JOINKEYS FILE=F2, FIELDS=(1, 7, A)
```

● SORTED

- 入力データセットがTSAMデータセットの場合、データセットのレコード処理方法を指定します。
- VはレコードにRDWを含め、F(デフォルト)の場合はレコードにRDWを含みません。
- 以下は、SORTJNF1とSORTJNF2を結合する例です。SORTJNF1はTSAMデータセットであり、レコード処理タイプはVです。

```
JOINKEYS FILE=F1, FIELDS=(5, 7, A), TYPE=V
JOINKEYS FILE=F2, FIELDS=(1, 7, A)
```

● TYPE

- 入力データセットがすでに結合フィールドを基準にソートされている場合は、このオペランドを指定することをお勧めします。
- 以下は、SORTJNF1とSORTJNF2を結合する例です。SORTJNF1はレコードの1番目のバイトから7バイトまでを基準にすでにソートされています。

```
JOINKEYS FILE=F1,FIELDS=(1,7,A),SORTED
JOINKEYS FILE=F2,FIELDS=(1,7,A)
```

- INCLUDE

- 入力レコードのうち、指定された条件を満たすレコードのみを使用して結合を実行します。logical_expressionの詳細については、[INCLUDE文](#)を参照してください。
- 以下は、SORTJNF1とSORTJNF2を結合する例です。SORTJNF1のレコードのうち、9番目のバイトが'A'のレコードのみ結合に参加します。

```
JOINKEYS FILE=F1,FIELDS=(1,7,A),INCLUDE=(9,1,CH,EQ,C'A')
JOINKEYS FILE=F2,FIELDS=(1,7,A)
```

- OMIT

- 入力レコードのうち、指定された条件を満たすレコードを除く残りのレコードのみを使用して結合を実行します。logical_expressionの詳細については、[OMIT文](#)を参照してください。
- 以下は、SORTJNF1とSORTJNF2を結合する例です。SORTJNF1のレコードのうち、9番目のバイトが'A'でないレコードのみ結合に参加します。

```
JOINKEYS FILE=F1,FIELDS=(1,7,A),OMIT=(9,1,CH,EQ,C'A')
JOINKEYS FILE=F2,FIELDS=(1,7,A)
```

2.9. MERGE

SORTINnn (nnは任意の2桁の数字であり、最初の桁に0を使用しても構いません) に指定した複数の入力データセットを出力データセットにマージするときに使用します。SORTINnnは、最大99まで使用可能であり、入力データセット内のレコードがMERGE文のFIELDSオプションに記述されたキー・フィールドに従ってソートされている必要があります。

OUTFILのFTOV、VTOFオプションを使用しない場合は、入力データセットのRECFMと出力データセットのRECFMが一致する必要があります。

構文

```
MERGE FIELDS=(p,m,f,s[,p,m,f,s[,...]]) | (p,m,s[,p,m,s[,...]]),FORMAT=f | COPY
[SKIPREC | STOPAFT | CKPT | EQUALS | NOEQUALS | FILES=n | FILSZ=[E|U]n |
SIZE=[E|U]n | Y2PAST | CENTWIN]
```

オペランド

以下は、オペランドについての説明です。

- FIELDS

- 入力レコードをマージするためのキー・フィールドを指定します。
- キーフィールドの開始位置と長さ、形式、マージの基準を指定します。
- 以下は、設定値についての説明です。

項目	説明
p	制御フィールドの開始位置を指定します。詳細については、 「2.16. SORT」 のFIELDSオペランドのp項目を参照してください。
m	制御フィールドの長さを指定します。
f	制御フィールドの形式を指定します。指定可能な形式と形式に応じた長さは、ソートの場合と同様です。
s	制御フィールドのソート基準を指定します。 - A(Ascending order) : 昇順 - D(Descending order) : 降順

- 以下は、形式が文字である固定長レコードの1番目のバイトから始まり、7バイトまでの内容を昇順にマージした例です。

```
MERGE FIELDS=(1, 7, CH, A)
```

以下は、可変長レコードの場合、1から4までのRDWを除き、実際のレコードの先頭である5から3バイト分の文字形式のレコードを降順にマージした例です。

```
MERGE FIELDS=(5, 3, CH, D)
```

参考

可変長レコードの場合、RDWを除いたレコードの先頭は5です。マージ開始点を1から4までの間で指定した場合、エラーは発生しませんが、RDWが含まれるのでマージが正常に行われません。

● FORMAT=f

- 制御フィールドの形式がすべて同じである場合は、各制御フィールド内で形式を指定せずに、外部からFORMAT=fオペランドを使用して一度に指定することができます。
- 以下はFORMATの例であり、2つのCH形式の固定長制御フィールドを指定しています。レコードの最初の1バイトから10バイト分は昇順にソートし、11バイトから10バイト分は降順にマージします。

```
MERGE FIELDS=(1, 10, A, 11, 10, D), FORMAT=CH
```

参考

OpenFrameでFORMAT=fオペランドを指定するときは、制御フィールドの形式がすべて同じである必要があります。たとえば、複数の制御フィールドのうち1つのフィールド形式が異なり、残りはすべて同じである場合、形式が異なるフィールドは内部で指定し、それ以外のフィールドの形式は外部からFORMAT=fで指定するとエラーが発生します。

● COPY

- COPYコマンドは、入力データセット・レコードの内容を制御フィールドの指定なしで出力データセットにコピーします。
- 以下は、COPYの例です。

```
MERGE FIELDS=COPY
```

参考

マージでCOPY入力項目を指定した場合、複数のSORTINを使用することができません。ソートでコピーを実行するのと同様、入力レコードをそのまま出力するためにマージ機能は実行しません。

● SKIPREC=x

- 入力レコードの1番目のレコードから始まり、ソートせずに削除(discard)するレコード数を指定します。
- xはレコードの数を意味します。
- SKIPRECはCOPYコマンドを使用する場合のみ使用できます。
- 以下はSKIPRECの例です。入力データセットの1番目のレコードから5番目のレコードまでを除いたレコードを出力データセットにコピーします。

```
MERGE FIELDS=COPY, SKIPREC=5
```

● STOPAFT=x

- 入力レコードの1番目のレコードから始まり、マージするレコード数を指定します。指定された数を越えたレコードは削除(discard)されます。
- xはレコードの数を意味します。
- STOPAFTはCOPYコマンドで使用する場合のみ使用できます。
- 以下はSTOPAFTの例です。入力データセットの1番目のレコードから5番目のレコードまでをコピーして出力し、残りのレコードは削除します。

```
MERGE FIELDS=COPY, STOPAFT=5
```

- CKPT

- チェックポイントを指定します。
- OpenFrameでは構文解析のみサポートします。

- EQUALS | NOEQUALS

- 制御フィールドの内容がすべて一致するレコードが入力された場合の出力順を指定します。
- 以下は、各項目についての説明です。

項目	説明
EQUALS	入力順に出力されます。（安定なソート）
NOEQUALS	入力順に出力されることを保証しません。（不安定なソート）（デフォルト値）

- 以下はEQUALS/NOEQUALSの例です。制御フィールドの内容に従ってソートが行われますが、制御フィールドの内容が一致するレコードの場合は、入力順に出力されます。

```
MERGE  FIELDS=(1, 7, CH, A), EQUALS
```

- FILES=n

- マージの入力ファイル数を10進数で指定します。
- OpenFrameでは構文解析のみサポートします。

- FILSZ=[E|U]n

- 実際にソートに使われるレコードの数を指定します。
- OpenFrameでは構文解析のみサポートします。

- SIZE=[E|U]n

- 入力レコードの数を指定します。
- 詳細については、[「2.11. OPTION」](#)を参照してください。

- Y2PAST | CENTWIN

- 詳細については、[「2.11. OPTION」](#)を参照してください。

2.10. MODS

MODS制御文を使用したスクリプトを使用してユーザー出口関数(user exit function)を登録することができます。

ユーザー関数は、前のフェーズから渡されたレコードを編集して、次のフェーズに渡します。ソートおよびコピーの演算では、ユーザー関数E15を登録して入力されるレコードを編集することができ

まず、ユーザー関数E35はOUTREC結果レコードを編集することができます。マージ演算はE32で処理しますが、OpenFrameではサポートされません。

構文

```
MODS exit=(n1,m1[,s1][,L1]),...,exit=(nn,mn[,sn][,Ln])
```

オペランド

以下は、オペランドについての説明です。

- exit
 - 登録するユーザー関数の名前を指定します。
 - OpenFrameではE15とE35のみサポートしています。

関数	説明
E15	ソートで入力レコードを処理するユーザー関数です。指定された入力データセットからレコードを読み込んで、ユーザーが指定した場所に渡します。
E35	出力レコードを処理するユーザー関数です。ソートまたはマージの後、出力データセットに渡される前のレコードを制御します。

- 以下は、設定項目についての説明です。

項目	説明
n	ユーザー関数のライブラリ内でのメンバー名を指定します。
m	ユーザー関数が使用している保存領域のサイズをバイト単位で保存します。OpenFrameではサポートされません。
s	ユーザー関数が入カストリーム内にある場合は、SYSINを指定します。
s	ユーザー関数が存在しているライブラリのDD名を指定します。 このオペランドを省略すると、STEPLIBからユーザー関数を検索します。JCLにSTEPLIBが記述されていない場合は、OpenFrame環境設定の sort サブジェクトの PATH セクションの EXITLIB キーに指定されたディレクトリから検索します。OpenFrame環境設定の詳細については、『OpenFrame 環境設定ガイド』を参照してください。
L	ユーザー関数に対してリンケージ・エディターが必要かどうかを指定します。OpenFrameではサポートされません。

例

以下は、ユーザー関数の例です。すべてのレコードをそのまま渡します。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int
EXIT(int *record_flags,
     char *new_rec,
     char *return_rec,
     char *unused1,
     int *unused2,
     int *new_rec_len,
     int *return_rec_len,
     int *unused3,
     int *exitarea_len,
     char *exitarea)
{
    if (*record_flags == 8)
        return 8; /* do not return */

    return 0; /* no action */
}
```

以下は、すべてのレコードをコピーして挿入するユーザー関数の例です。レコードa、b、cが前のフェーズから渡されると、a、a、b、b、c、cの形式で次のフェーズに渡されます。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int
INSERT(int *record_flags,
       char *leaving_rec,
       char *return_rec,
       char *output_rec,
       int *unused1,
       int *leaving_rec_len,
       int *return_rec_len,
       int *output_rec_len,
       int *exitarea_len,
       char *exitarea)
{
    static int inserted = 0;
    /* 挿入されたレコードは編集しない */
}
```

```

    if (inserted) {
        inserted = 0;
        return 0; /* no action */
    }

    if (*record_flags == 8)
        return 8; /* do not return */

    memcpy (return_rec, leaving_rec, *leaving_rec_len);
    *return_rec_len = *leaving_rec_len;
    inserted = 1;

    return 12; /* insert record */
}

```

以下は、E15にEXITというメンバー名の関数を登録し、E35にINSERTというメンバー名の関数を登録する例です。動的にリンキングされるライブラリ・ファイルはSYS1.JCLLIBディレクトリに存在しており、このディレクトリはMYLIBというDD名で指定されています。

```

MYLIB DD DSN=SYS1.JCLLIB,DISP=OLD
...
MODS E15=(EXIT,3990,MYLIB),
      E35=(INSERT,3990,MYLIB)

```

2.11. OPTION

OPTION文は、各種オプションを設定します。

構文

```

OPTION COPY | MSGPRT | PRINT | SIZE=[E|U]n | FILSZ=[E|U]n | CKPT |
        DYNALLOC=[([d][,n])] | OVFL0=(RC0|RC4|RC16) | SOLRF | NOSOLRF |
        SORTDD=cccc | SORTIN=ddname | SORTOUT=ddname | ZDPRINT | NZDPRINT |
        Y2PAST=(s|f) | CENTWIN=value | NULLOUT=(RC0|RC4|RC16) |
        VLSHRT | NOVLSHRT | VLTESTI=(0|1|2) |
        USE_ALTERNATIVE_TP_TYPE=(Y|N)

```

オプション

制御フィールドの内容を記述した後、以下のオプションを使用することができます。

- COPY
 - SORTINデータセットを出力します。

- SORT FIELDS=COPYと同じ機能です。
- 同オプションを使用すると、SORT文は無視されます。

参考

OpenFrameでのOPTION COPYは、SORT文またはMERGE文と一緒に使用できません。

- MSGPRT
 - メッセージの出力対象を指定します。OpenFrameでは構文解析のみサポートします。
- PRINT
 - MSGPRTと同様な機能です。OpenFrameでは構文解析のみサポートします。
- SIZE=[E|U]n
 - 入力レコードの数がこのオプションに指定された値(n)と異なる場合はエラーが発生します。
 - OpenFrame環境設定の**sort**サブジェクト、**OPTION**セクションの**FSZEST**キーの値がYESに指定されていたり、あるいはnの前にEまたはUが付いていると、このオプションは無視されます。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
- FILSZ=[E|U]n
 - 実際にソートに使われるレコードの数を指定します。
 - OpenFrameでは構文解析のみサポートします。
- CKPT
 - コピーの使用時にチェックポイントを指定します。
 - OpenFrameでは構文解析のみサポートします。
- DYNALLOC [=([d],[n])]
 - 処理用データセットを動的に割り当てます。
 - 以下は、設定項目についての説明です。

項目	説明
d	動的に割り当てるデバイスのタイプまたはグループ名を指定します。
n	動的に割り当てる処理用データセットの数を指定します。

- OpenFrameでは構文解析のみサポートします。
- OVFO=[RC0 | RC4 | RC16]
 - sumフィールドでオーバーフローが発生した場合の動作を設定します。
 - 以下は、設定項目についての説明です。

項目	説明
RC0	オーバーフローが発生した場合、警告メッセージを1回出力し、戻り値として0を返します。
RC4	オーバーフローが発生した場合、警告メッセージを1回出力し、戻り値として4を返します。
RC16	オーバーフローが発生した場合、エラーを出力し、戻り値として16を返します。

- SOLRF|NOSOLRF

- SORTOUTレコードの長さが指定されていない場合に従う長さを設定します。
- 以下は、設定項目についての説明です。

区分	説明
SOLRF	再フォーマットされたレコード長に従います。レコード長を指定する際の優先順位は、 「1.3.2. データセットの定義」 を参照してください。
NOSOLRF	再フォーマットされたレコード長に従いません。 レコード長を指定する際の優先順位は以下のとおりです。 1. RECORD文のL3に指定された値 2. RECORD文のL2にE15で指定された値 3. SORTINに指定された最大長 4. RECORD文のL1に指定された値

- SORTDD=cccc

- SORTIN、SORTnn、SORTOUT、SORTWKnnなどのデータセットのDD名の先頭4桁の文字「SORT」を代替する4桁の文字列を設定します。

- SORTIN=ddname

- SORTINデータセットとして使用するDD名を8桁以下の文字列で設定します。

- SORTOUT=ddname

- SORTOUTデータセットとして使用するDD名を8桁以下の文字列で設定します。

- ZDPRINT | NZDPRINT

- ZD形式フィールドをEBCDIC形式で表現するオプションです。OpenFrameでは、ZD形式フィールドをASCII形式で出力するため、このオプションは不要です。
- OpenFrameでは構文解析のみサポートします。

- Y2PAST=[s | f]

- 一時的にsliding(s)またはfixed(f)世紀ウィンドウでY2PASTオプションを上書きします。
世紀ウィンドウは、Y2形式と一緒に2桁の年のデータ値を4桁の年のデータ値に正しく解釈するために使用されます。
- 以下は、設定項目についての説明です。

設定値	説明
s	現在の年からs値を減算した年が、スライディング世紀ウィンドウの開始値となります。 たとえば、Y2PAST=11で現在の年が2011の場合、世紀ウィンドウは2000～2099となります。
f	固定世紀ウィンドウの開始値を指定します。 たとえば、Y2PAST=2011の場合、世紀ウィンドウは2011～2110となります。

- CENTWIN=value
 - Y2PAST=valueと同様な機能です。
- NULLOUT=[RC0 | RC4 | RC16]
 - SORTOUTデータセットに書き込むレコード数が0である場合の動作を設定します。
 - 以下は、設定項目についての説明です。

設定値	説明
RC0	戻り値として0を返します。
RC4	戻り値として4を返します。
RC16	戻り値として16を返します。

- VLSHRT|NOVLSHRT
 - SORT、MERGE、INCLUDE、OMIT、SUMなどの演算で、指定されたフィールドが入力レコードに存在しないか、一部のみ存在する場合の動作を定義します。
 - 以下は、設定項目についての説明です。

設定値	説明
VLSHRT	SORTまたはMERGE演算の場合、指定されたフィールドがレコードに存在しないか一部のみ存在する場合は、ヌル文字が埋め込まれます。 – INCLUDEまたはOMIT演算：指定されたフィールドが入力レコードに存在しないか、一部のみが存在する場合は、関係式は「偽」と見なされます。

設定値	説明
	– SUM演算: 指定されたフィールドが入力レコードに存在しないか、一部のみが存在する場合は、そのレコードを演算から除外します。
NOVLSHRT	指定されたフィールドが入力レコードに存在しないか、一部のみが存在する場合は、エラーが発生します。

● VLTESTI=(0|1|2)

- INCLUDEまたはOMIT演算で、指定されたフィールドが入力レコードに存在しないか、一部のみ存在する場合の動作を定義します。
- 以下は、設定項目についての説明です。

設定値	説明
0	INCLUDEまたはOMIT演算は、指定されたすべてのフィールドの全範囲が入力レコードに存在する必要があります。そうでない場合は、エラーが発生します。VLTESTIオプションが指定されていない場合は、VLTESTI=0が指定されているかのように動作します。
1	INCLUDE演算は、指定されたフィールドが入力レコードに存在しないか、一部のみ存在する場合は、レコードを出力しません。OMIT演算は、指定されたフィールドが入力レコードに存在しないか、一部のみ存在する場合、レコードを出力します。
2	INCLUDEまたはOMIT演算は、指定されたフィールドが入力レコードに存在しないか、一部のみ存在する場合、関係式は「偽」と見なされます。

2.12. OUTFIL

SORTOUTをOUTFIL DDに指定した複数のデータセットに保存します。レコードをフィルタリングして再フォーマットするために使用できます。各出力データセットに同じレコードを出力することが可能であり、各出力レコードで異なる編集を行い、それぞれ異なるレコードを出力することもできます。(別名: OUTFILE)

構文

```
OUTFIL param[,param[,...]]
param = <FNAME=ddname[,ddname[,...]]> |
        <FILES=suffix[,suffix[,...]]> | STARTREC | ENDREC | INCLUDE | OMIT |
        SAVE | OUTREC | PARSE | BUILD | OVERLAY | FINDREP | IFTHEN | SPLIT |
        SPLITBY | SPLIT1R | VTOF | CONVERT | VLFILL=c | FTOV | VLTRIM=c |
```

オペランド

以下は、オペランドについての説明です。

- FNames=ddname|(ddname,ddname,...)

- OUTFILの結果が保存されるDD名の識別子を指定します。
- 他のOUTFIL文の識別子を重複して使用することはできません。オペランドが省略された場合、出力データセットのDD名はSORTOUTと指定されます。
- 複数のデータセットに同じ処理をして同じレコードを出力する場合、たとえば、SORTOF01、SORTOF02、SORTOF03に同じ内容を出力する場合は、以下のように指定できます。

```
FNames=(SORTOF01,SORTOF02,SORTOF03)
```

- FILES=suffix

- OUTFILの結果が保存されるDD名の識別子を指定します。
- 複数のレコードを出力する場合、prefixに「SORTOF」がデフォルトとして指定され、FILESオペランドを使用して1文字または2文字の識別子を添付します。他のOUTFIL文の識別子を重複して使用することはできません。オペランドが省略された場合、出力データセットのDD名はSORTOUTと指定されます。
- 複数のデータセットに同じ処理をして同じレコードを出力する場合、たとえば、SORTOF01、SORTOF02、SORTOF03に同じ内容を出力する場合は、以下のように指定できます。

```
FILES=(01,02,03)
```

- STARTREC=n

- 入力レコードでOUTFIL動作が開始する時点です。

- ENDREC=n

- 入力レコードでOUTFIL動作が終了する時点です。

- INCLUDE | OMIT

- OUTFIL処理中に条件を満たすレコードのみを出力するか(INCLUDE)、条件を満たすレコードのみを除外するか(OMIT)を指定します。
- INCLUDE/OMIT制御文では、「INCLUDE/OMIT COND=」形式で指定しますが、OUTFILでは「INCLUDE/OMIT=」形式で条件を指定します。
- FORMAT=fオペランドで形式を指定できないことを除いては、INCLUDE/OMIT制御文の使用方法と同じです。

- SAVE

- INCLUDEとOMITを実行した後、上記のフィルターに該当しないレコードを出力します。

- PARSE

- 可変長入力レコードから解析済みの固定長フィールドを抽出します。解析済みフィールドは、BUILDまたはOVERLAYの(p,m)を使用できる場所で使用できます。
- 以下は、オプションについての説明です。

オプション	説明
%nn	可変長入力レコードを固定長出力レコードに変換します。 解析済みフィールドは、BUILDまたはOVERLAYの(p,m)を使用できる場所で使用できます。
FIXLEN=m	解析済みフィールドの長さを指定します。この属性は、すべての解析済みフィールドに存在する必要があります。
ABSPOS=p	入力レコードで解析開始位置のpを指定します。
ADDPOS=x	入力レコードで解析開始位置がxだけ増加します。
SUBPOS=y	入力レコードで解析開始位置がyだけ減少します。
STARTAFT	– STARTAFT=string : 入力レコードの最初の文字列の後から解析済みフィールドのレコードを開始します。 – STARTAFT=BLANKS : 入力レコードの最初のブランク・グループの後から解析済みフィールドのレコードを開始します。
STARTAT	– STARTAT=string : 入力レコードの最初の文字列から解析済みフィールドのレコードを開始します。 – STARTAT=BLANKS : 入力レコードの最初のブランクから解析済みフィールドのレコードを開始します。 – STARTAT=NONBLANK : 入力レコードの最初の非ブランクから解析済みフィールドのレコードを開始します。
ENDBEFR	– ENDBEFR=string : 入力レコードの最初の文字列の前のバイトで解析済みフィールドのレコードを終了します。 – ENDBEFR=BLANKS : 入力レコードの最初のブランクの前のバイトで解析済みフィールドのレコードを終了します。
ENDAT	– ENDAT=string : 入力レコードの最初の文字列の最後のバイトで解析済みフィールドのレコードを終了します。 – ENDBEFR=BLANKS : 入力レコードの最初のブランク・グループの終わりで解析済みフィールドのレコードを終了します。

オプション	説明
PAIR	- PAIR=APOST : アポストロフィ(')で囲まれた領域は、文字列またはブランクとして扱われません。 - PAIR=QUOTE : 引用符(")で囲まれた領域は、文字列またはブランクとして扱われません。

● OUTREC | BUILD

- OUTFIL処理中にレコード・フィルター(INCLUDE/OMIT)を通過したレコードをOUTFIL DDに記録する前に、指定された条件に合わせてレコードを再フォーマットします。「OUTREC COND=」と指定せず、「OUTREC=」と指定することを除いては、OUTREC制御文の使用方法と同じです。
- OUTRECの代わりにBUILDを使用することができます。
- 以下は、オプションについての説明です。

オプション	説明
c:	出力レコードの開始位置を指定します。
s	分離レコードとして文字列や空白を指定します。 - nX : n個の空白文字を挿入します。 - nC'xx...x' : 文字列を挿入します。nは文字列の繰り返し回数です。 - nX'yy...yy' : 16進数の文字列を挿入します。nは文字列の繰り返し回数です。
DATEn	現在の日付を出力します。OpenFrameではDATE1のみサポートしており、「C'yyyymmdd」形式で表示されます。
DATEn+r	未来の日付を出力します。 OpenFrameではDATE1+rのみサポートしており、現在の日付からr日を加算した日付を出力します。
TIME n	現在の時間を出力します。 nは1~3の範囲で指定可能であり、以下の形式で表示されます。 1 : C'hhmmss' 2 : C'hhmm' 3 : C'hh'
DATE=(abcd) または &DATE=(abcd)	現在の日付を「C'adbdc」形式で出力します。dは区切り文字です。 a,b,c∈{M,D,Y,4} であり、それぞれの意味は以下のとおりです。 M : 月(01~12)を2桁の数字で表示します。

オプション	説明
	– D : 日(01~31)を2桁の数字で表示します。 – Y : 年の下2桁を表示します。 – 4 : 年を4桁の数字で表示します。
DATE または &DATE	現在の日付を「C'mm/dd/yy」形式で出力します。 – mm : 月(01-12) – dd : 日(01-31) – yy : 年の下2桁
DATENS=(abc) または &DATENS=(abc)	現在の日付を「C'abc」形式で出力します。 a,b,c∈{M,D,Y,4} であり、それぞれの意味は以下のとおりです。 – M : 月(01~12)を2桁の数字で表示します。 – D : 日(01~31)を2桁の数字で表示します。 – Y : 年の下2桁を表示します。 – 4 : 年を4桁の数字で表示します。
/.../ or n/	「/」の数またはn個分の新規行を挿入します。 現在OpenFrameでは、「/.../」の場合は5つまでのみサポートしており、「n/」の場合は制限がありません。
p	指定された入力レコードのフィールドを出力フィールドに出力します。 入力レコードは、場合に応じて開始時点が異なります。 – 入力レコードが非VSAM固定長(FB)の場合、入力レコードの1番目のバイトが1となります。 – 入力レコードが非VSAM可変長(VB)の場合、レコードの前に4バイトのレコード記述語(RDW)が追加されるので、RDWの開始点が1となり、実際のレコードは5から始まります。 – 入力レコードがVSAMデータセットであり、OpenFrame SORTでこれを可変長レコードとして処理する場合、RDWをレコードの前に追加し、その開始点を1とします。したがって、結合を実行する際に実際のデータは5から始まります。 また、OpenFrame SORTの実行時に、VSAMデータセットを固定長レコードと判断するか、あるいは可変長レコードと判断するかを決める必要があります。以下のいずれかの条件を満たす場合、そのデータセットは可変長レコードと判断されます。

オプション	説明
	1. RECORD TYPE=Vと指定されている場合 2. SORTOUTがRECFM=VBと指定されている場合 3. SORTINとSORTOUTが両方ともVSAMの場合 VBレコードの場合、最初のキー・フィールドを指定する際は常にpを1に指定する必要がある、最初のキー・フィールドはRDWを含む必要があります。1番目のフィールドを1から4まで指定した後次のフィールドを指定するか、1番目のフィールドを1から4以上に指定することができます。ただし、後者の場合、実際のレコードは5から始まるという点に注意します。
m	指定された入力レコードのフィールドを出力フィールドに出力します。選択フィールドの長さをバイト数で指定します。
%nn	%nnラベルの解析済みレコード・フィールドを出力フィールドに出力します。
p,m,TRAN=LTOU UTOL	– p, m : 前述の項目説明を参照してください。 – TRAN=LTOU : フィールドの小文字を大文字に変換して出力します。 – TRAN=UTOL : フィールドの大文字を小文字に変換して出力します。
p,m,TRAN=ALTSEQ	– p, m : 前述の項目説明を参照してください。 – TRAN=ALTSEQ : フィールドのレコードをALTSEQ変換表に基づいて変更します。
p,m,Y2x	– p : 前述の項目説明を参照してください。 – m : 年を示す出力フィールドの長さを指定します。 – Y2x : OpenFrameではY2Cのみサポートしており、「C'yyyy'」形式で出力されます。
p,m,DTx	– p : 前述の項目説明を参照してください。 – m : 指定された入力レコード・フィールドを出力フィールドに出力します。フィールドの長さは、バイト数で指定します。入力レコード・フィールドの形式は、「X'0cyydddF'」です。(cは、20世紀の場合は0、21世紀の場合は1です。yyは2桁の年、dddは年間通算日を示します) – DT1 : 入力レコード・フィールドを「C'yyyymmdd'」形式で出力します。yyyyは4桁の年、mmは月、ddは日です。 – DT2 : 入力レコード・フィールドを「C'yyyymm'」形式で出力します。yyyyは4桁の年、mmは月です。

オプション	説明
	DT3: 入力レコード・フィールドを「C'yyyyddd」形式で出力します。yyyyは4桁の年、dddは年間通算日です。
p,m,TMx	- p: 前述の項目説明を参照してください。 - m: 指定された入力レコード・フィールドを出力フィールドに出力します。フィールドの長さはバイト数で指定します。入力レコード・フィールドの形式は、「X'nnnnnnnnF」です。(nnnnnnnnは、午前0時からの経過時間を100分の1秒単位で表した値です) - TM1: 入力レコード・フィールドを「C'hhmmss」形式で出力します。hhは時、mmは分、ssは秒です。 - TM2: 入力レコード・フィールドを「C'hhmm」形式で出力します。hhは時、mmは分です。 - TM3: 入力レコード・フィールドを「C'hh」形式で出力します。hhは時です。 - TM4: 入力レコード・フィールドを「C'hhmmssxx」形式で出力します。hhは時、mmは分、ssは秒、xxは100分の1秒です。
p,m,f,edit	指定されたフィールドを定義された形式に変更してOUTFILレコードに出力します。 - p, m: 前述の項目説明を参照してください。 - f: フィールドの形式を指定します。 - edit: フィールドが変換される形式を定義します。EDITコマンドの詳細については、EDITコマンドを参照してください。
p,m,f,to	入力フィールドの形式を変更して出力します。 - p,m,f: 前述の項目説明を参照してください。 - to: 入力フィールドの形式が変更される出力レコードの形式を指定します。 入力フィールドの形式がZDFの場合、fがZDの場合と同様に動作します。 toコマンド構文は以下のとおりです。 <pre>of TO=[of (of)] [, LENGTH=[n (n)]]</pre> - TO=[of (of)]: 変更される形式を定義します。「TO=」を省略して使用することもできます。 - LENGTH=[n (n)]: 変更される出力フィールドの長さを定義します。

オプション	説明
arexp,edit	算術式(arithmetic expression)を編集形式に変更してOUTFILレコードに出力します。 算術式を意味しており、形式は以下のとおりです。 <pre>p,m,f,operator,n[,operator,n,...]</pre> – p, m, f : 前述の項目説明を参照してください。 – n : 10進数であり、正数と負数のいずれも使用が可能です。 pからm分のf形式の数を、演算子を使用してnと演算します。 以下は、演算子の種類です。 – MIN(minimum) – MAX(maximum) – MUL(multiplication) – DIV(division) – MOD(modulus) – ADD(addition) – SUB(subtraction) 以下は、演算子の優先順位です。 1. MIN、MAX 2. MUL、DIV、MOD 3. ADD、SUB
p,m,CHANGE	入力フィールドの指定された文字列を他の文字列に置換して出力します。 – p : 前述の項目説明を参照してください。 – m : findに指定された文字列と比較する入力フィールドの長さを指定します。 – CHANGE : 比較する文字列を検索し、同じものがあれば置換します。 CHANGEの構文は以下のとおりです。 <pre>CHANGE=(v,find,set[,find,set,...]),NOMATCH=(set)</pre> • v : 出力するフィールドの長さを指定します。 • find : 入力フィールドで検索する文字列を指定します。

オプション	説明
	• set : 出力フィールドに出力する文字列を指定します。 • NOMATCH : findに指定した文字列が存在しない場合、NOMATCH オプションを使用します。setにはfindと一致する文字列がない場合に出力する文字列を指定します。
seqnum	出力レコードのシーケンス番号を出力します。シーケンス番号は、入力レコードの後に追加されます。 以下は、OpenFrameでサポートされる形式です。 <pre>SEQNUM, n, fs[, START=j][, INCR=i][, RESTART=(p, m)]</pre> – SEQNUM : シーケンス番号を出力するコマンドを指定します。 – n : シーケンス番号の桁数を指定します。必須オプションです。 – fs : シーケンス番号の文字列形式を指定します。必須オプションです。 – START=j : シーケンス番号の開始値をjに設定します。このオプションは省略可能であり、省略した場合は、START=1と見なされます。 – INCR=i : シーケンス番号の増分値をiに設定します。このオプションは省略可能であり、省略した場合は、INCR=1と見なされます。 – RESTART=(p,m) : このオプションは省略できます。指定されたフィールドが前のレコードのフィールドと一致しない場合、SEQNUM はjに初期化されます。
p,m,justify	指定されたフィールドを定義された形式に変換してOUTFILレコードとして出力します。 以下は、OpenFrameでサポートされる形式です。 <pre>JFY=(SHIFT=(LEFT RIGHT), [LENGTH=n], [PREBLANK=list], [LEAD=string], [TRAIL=string])</pre> – SHIFT : 左のブランクを削除した後、左揃えにするか、右のブランクを削除した後、右揃えにするかを指定します。 – LENGTH=n : JFYの出力フィールドの長さを指定します。指定しない場合は、入力フィールドの長さと同じです。 – PREBLANK=list : 1つ以上の文字列リストをブラックに変換します。 – LEAD=string : 最初の非ブラック・フィールドの前に文字列を指定します。 – TRAIL=string : 最後の非ブラック・フィールドの後に文字列を指定します。

オプション	説明
%nn,justify	%nnラベルの解析済みレコード・フィールドを定義された形式に変換して出力します。
p,m,squeeze	指定されたフィールドを定義された形式に変換してOUTFILレコードとして出力します。 以下は、OpenFrameでサポートされる形式です。 <pre>SQZ=(SHIFT=(LEFT RIGHT), [LENGTH=n], [PREBLANK=list], [LEAD=string], [MID=string], [TRAIL=string], [PAIR=(APOST QUOTE)])</pre> – SHIFT: すべてのブラックを削除した後、左揃えにするか右揃えにするかを指定します。 – LENGTH=n: SQZの出力フィールドの長さを指定します。指定しない場合は、入力フィールドの長さと同じです。 – PREBLANK=list: 1つ以上の文字列リストをブラックに変換します。 – LEAD=string: 最初の非ブラック・フィールドの前に文字列を指定します。 – MID=string: 最初と最後の非ブランク・フィールドの間のブランクが削除される領域に文字列を指定します。 – TRAIL=string: 最後の非ブランク・フィールドの後に文字列を指定します。 – PAIR=(APOST QUOTE): 単一引用符(')または二重引用符("")の間のレコード値は、ブランクを削除するか、あるいはPREBLANKを使用してブランクに変換することができません。
%nn,squeeze	%nnラベルの解析済みレコード・フィールドを定義された形式に変換して出力します。

- OVERLAY

- 出力レコードに上書きする項目を指定します。上書きされていない列はそのまま出力します。

参考

OUTREC/BUILDで使用するパラメータを同様に使用できますが、「/」文法は使用できません。

- FINDREP

- 入力レコードで指定された変数を検索し、他の変数に置換します。
- 入力レコードでinconに該当する文字列を検索し、outconに指定された文字列に置換します。

```
IN=incon,OUT=outcon
```

- 入力レコードで複数の文字列を指定し、outconに置換できます。

```
IN=(incon1,incon2[,...]),OUT=outcon
```

- 入力レコードでincon_iに該当する文字列を検索し、outcon_iに指定された文字列に置換します。(i ≥ 1)

```
INOUT=(incon1,outcon1[,incon2,outcon2[,...]])
```

- 入力レコードで検索の開始点を指定できます。STARTPOSを指定していない場合は1に指定されます。

```
STARTPOS=p
```

- 入力レコードで検索の終了点を指定できます。ENDPOSを指定していない場合はレコードの最後の列まで検索します。

```
ENDPOS=q
```

● IFTHEN

- 指定された条件に従って、各レコードを異なる方法で再フォーマットできます。
- BUILD、OVERLAY、FINDREPの再フォーマット機能を適用する前に1回以上使用できます。WHEN=GROUPは、IFTHEN=INIT以外の他の項目より先に実行されます。

```
WHEN=GROUP,BEGIN=(logexp),END=(logexp),RECORDS=n,PUSH=(items)
```

項目	説明
BEGIN=(logexp)	logexp条件が満たされると、新しいグループが開始されます。
END=(logexp)	logexp条件が満たされると、現在のグループが終了します。
RECORDS=n	グループに属するレコードの数がnに達すると、グループを終了します。
PUSH=(items)	- c:p,m: グループの最初のレコードのp、mの値をcの位置にオーバーレイします。 - c:ID=n: 長さnのグループIDを表します。現在のグループIDを位置cにオーバーレイします。 - c:SEQ=n: 長さnのグループのレコード・シーケンスを表します。現在のグループのレコード・シーケンスを位置cにオーバーレイします。

- BUILD、OVERLAY、FINDREPの再フォーマット機能を適用するために1回以上使用できます。WHEN=GROUPは、IFTHEN=GROUP以外の他の項目より先に実行されます。

WHEN=INIT,BUILD/OVERLAY/FINDREP

- 入力レコードがlogexpに指定された論理式を満たす場合、BUILD、OVERLAY、FINDREPの再フォーマット機能を適用するために1回以上使用できます。

WHEN=(logexp), BUILD/OVERLAY/FINDREP

- 入力レコードがlogexpに指定された論理式をすべて満たさない場合、OVERLAY、FINDREPの再フォーマット機能を適用するために1回使用できます。

WHEN=NONE,BUILD/OVERLAY/FINDREP

- SPLIT | SPLITBY | SPLIT1R

- 再フォーマットされたレコードを複数のOUTFILデータセットに順次に出力します。

項目	説明
SPLIT	複数の出力データセットにレコードを順次に1つずつ割り当てて出力します。
SPLITBY=n	複数の出力データセットにレコードを順次にn個ずつ割り当てて出力します。 SPLIT機能はSPLITBY=1と同じです。
SPLIT1R=n	複数の出力データセットにレコードを順次にn個ずつ出力し、最後のデータセットに到達すると残りのレコードをすべて出力します。

- VTOF | CONVERT

- 可変長レコードを固定長レコードに変換します。

- VLFILL=c

- フィールドが入力レコードに存在しないか一部のみ存在する場合、右側を文字cで埋めます。

- FTOV

- 固定長レコードを可変長レコードに変換します。

- VLTRIM=c

- 結果レコードの右端に1つまたはそれ以上の文字cが存在する場合はそれを削除します。

- NULLOFL= [RC0 | RC4 | RC16]

- 演算結果のレコード数が0件の場合の動作を指定します。

項目	説明
RC0	戻り値を0に設定し、以降の演算を続けます。
RC4	戻り値を4に設定し、以降の演算を続けます。

項目	説明
RC16	演算を中止し、16を返します。

- LINES=n

- レポート・ページの行数を設定します。（デフォルト値: 60）

- HEADER1

- レポートのヘッダーを作成します。HEADER1はレポートの最初のページにのみ出力されます。
- 以下は、オプションについての説明です。

オプション	説明
c:	関連するレポート内容が出力される最初の位置を指定します。
r	空白または文字列を出力レポート内に挿入できます。 – nX : n個の空白文字を挿入します。 – n'xx...x' : 文字列をn個挿入します。 – nX'yy...yy' : 16進の文字列をn個挿入します。 – /.../ or n/ : '/'の数またはn個分の新規行を挿入します。現在OpenFrameでは、「/.../」の場合は5つまでのみサポートしており、「n/」の場合は制限がありません。
p,m	OUTFILの入力レコードのフィールドをレポートに出力します。 – p : 入力レコードの最初のバイトから相対的な位置です。 – m : 入力フィールドの長さを意味します。
DATE または &DATE	現在の日付を「C'mm/dd/yy」形式で出力します。 – mm : 月(01-12) – dd : 日(01-31) – yy : 年の下2桁
DATE=(abcd) または &DATE=(abcd)	– abc : a、b、cにM、D、Yを使用することができ、それぞれ月、日、年を示します。M、D、Yの順序によって出力の順序を決定します。Yは、4に代替することができます。その場合、年は4桁で表示されます。 – d : 区切り文字です。たとえば、DATE=(MDY-)のように使用する場合、「C'mm-dd-yy」形式で出力されます。
TIME または &TIME	現在の時間を「C'hh:mm:ss」形式で出力します。 – hh : 時間(00-23)

オプション	説明
	– mm : 分(00-59) – ss : 秒(00-59)
TIME=(abc) または &TIME=(abc)	時間を24時間制で表示するか、あるいは12時間制で表示するかを指定します。 – ab : 24または12を指定できます。24は24時間制、12は12時間制を示します。 – c : 区切り文字であり、「C'hhcmmcss'」形式で挿入されます。
PAGE または &PAGE	レポートの8桁のページ番号をヘッダーに1回のみ出力します。

● TRAILER1

- レポートのトレーラーを作成します。TRAILER1はレポートの最後のページにのみ出力されます。
- 以下は、各オプションについての説明です。

オプション	説明
c:	HEADER1オペランドを参照してください。
r	HEADER1オペランドを参照してください。
p,m	HEADER1オペランドを参照してください。
DATE	HEADER1オペランドを参照してください。
&DATE	HEADER1オペランドを参照してください。
TIME	HEADER1オペランドを参照してください。
&TIME	HEADER1オペランドを参照してください。
PAGE または &PAGE	現在のページ番号をトレーラーに1回のみ出力します。
COUNT	出力レコードの数を8桁で表示します。
COUNT15	出力レコードの数を15桁で表示します。
SUBCOUNT	入力レコードの現在までの累積カウントを出力します。 TRAILER1ではCOUNTと同様に動作します。
SUBCOUNT15	入力レコードの現在までの累積カウントを出力します。 TRAILER1ではCOUNT15と同様に動作します。
TOTAL=(p,m,f)	指定された数字入力フィールドのトータルを出力します。TOTとも使用することができます。

オプション	説明
	p、m、fは前述の項目説明を参照してください。
MIN=(p,m,f)	指定された数字入力フィールドでの最小値を出力します。 p、m、fは前述の項目説明を参照してください。
MAX=(p,m,f)	指定された数字入力フィールドでの最大値を出力します。 p、m、fは前述の項目説明を参照してください。
AVG=(p,m,f)	指定された数字入力フィールドの平均値を出力します。 p、m、fは前述の項目説明を参照してください。
SUBTOTAL=(p,m,f)	現在までの累積トータルを出力します。TRAILER1ではTOTALと同じ結果を出力します。省略形のSUBTOTまたはSUBも使用できます。
SUBMIN=(p,m,f)	現在までの累積最小値を出力します。 TRAILER1ではMINと同じ結果を出力します。
SUBMAX=(p,m,f)	現在までの累積最大値を出力します。 TRAILER1ではMAXと同じ結果を出力します。
SUBAVG=(p,m,f)	現在までの累積平均を出力します。 TRAILER1ではAVGと同じ結果を出力します。

- HEADER2

- レポートのヘッダーを作成します。HEADER2はレポートのページごとに出力されます。
- 以下は、各オプションについての説明です。

オプション	説明
c:	HEADER1オペランドを参照してください。
r	HEADER1オペランドを参照してください。
p,m	HEADER1オペランドを参照してください。
DATE	HEADER1オペランドを参照してください。
&DATE	HEADER1オペランドを参照してください。
TIME	HEADER1オペランドを参照してください。
&TIME	HEADER1オペランドを参照してください。
PAGE または &PAGE	ページ番号を各ページに8桁で出力します。

- TRAILER2

- レポートのトレーラーを作成します。TRAILER2はレポートのページごとに出力されます。
- 以下は、各オプションについての説明です。

オプション	説明
c:	HEADER1オペランドを参照してください。
r	HEADER1オペランドを参照してください。
p,m	HEADER1オペランドを参照してください。
DATE	HEADER1オペランドを参照してください。
&DATE	HEADER1オペランドを参照してください。
TIME	HEADER1オペランドを参照してください。
&TIME	HEADER1オペランドを参照してください。
PAGE	HEADER2オペランドを参照してください。
&PAGE	HEADER2オペランドを参照してください。
COUNT	TRAILER1オペランドを参照してください。
COUNT15	TRAILER1オペランドを参照してください。
SUBCOUNT	TRAILER1オペランドを参照してください。
SUBCOUNT15	TRAILER1オペランドを参照してください。
TOTAL	TRAILER1オペランドを参照してください。
MIN	TRAILER1オペランドを参照してください。
MAX	TRAILER1オペランドを参照してください。
AVG	TRAILER1オペランドを参照してください。
SUBTOTAL	TRAILER1オペランドを参照してください。
SUBMIN	TRAILER1オペランドを参照してください。
SUBMAX	TRAILER1オペランドを参照してください。
SUBAVG	TRAILER1オペランドを参照してください。

● SECTIONS

OUTFILによって作成されるレポートのセクションを区分します。指定されたフィールドのバイナリー値の変更に従ってセクションが分けられます。

- p,m
 - セクション区分の基準となるフィールドを指定します。詳細については、HEADER1オペランドを参照してください。

– SKIP=P | L | nL

- セクションをどのように区分して示すかを指定します。
- 以下は、設定項目についての説明です。

項目	説明
P	各セクションを新規ページに出力します。OpenFrameでは構文解析のみサポートしています。
L	1つの空白行を間に入れて、セクションを同じページに出力します。 Lは1Lと同じです。
nL	n個の空白行を間に入れて、セクションを同じページに出力します。

– HEADER3

- セクションのヘッダーを作成します。
- 以下は、オプションについての説明です。

オプション	説明
c:	HEADER1オペランドを参照してください。
r	HEADER1オペランドを参照してください。
p,m	HEADER1オペランドを参照してください。
DATE	HEADER1オペランドを参照してください。
&DATE	HEADER1オペランドを参照してください。
TIME	HEADER1オペランドを参照してください。
&TIME	HEADER1オペランドを参照してください。
PAGE または &PAGE	セクションごとに8桁のページ番号を出力します。

– PAGEHEAD

- セクション・ヘッダーがレポートの各ページの先頭に出力されるようにします。
- PAGEHEADが指定されていない場合、セクション・ヘッダーは各セクションの前にのみ出力されます。したがって、セクションが2ページにまたがっている場合、セクション・ヘッダーがページの間に出力されることがありますが、同オプションを使用するとセクション・ヘッダーを一番上に出力することができます。
- PAGEHEADはHEADER3がある場合にのみ使用できます。
- OpenFrameでは構文解析のみサポートします。

– TRAILER3

- セクションのトレーラーを作成します。
- 以下は、オプションについての説明です。

オプション	説明
c:	HEADER1オペランドを参照してください。
r	HEADER1オペランドを参照してください。
p,m	HEADER1オペランドを参照してください。
DATE	HEADER1オペランドを参照してください。
&DATE	HEADER1オペランドを参照してください。
TIME	HEADER1オペランドを参照してください。
&TIME	HEADER1オペランドを参照してください。
PAGE	HEADER2オペランドを参照してください。
&PAGE	HEADER2オペランドを参照してください。
COUNT	TRAILER1オペランドを参照してください。
COUNT15	TRAILER1オペランドを参照してください。
SUBCOUNT	TRAILER1オペランドを参照してください。
SUBCOUNT15	TRAILER1オペランドを参照してください。
TOTAL	TRAILER1オペランドを参照してください。
MIN	TRAILER1オペランドを参照してください。
MAX	TRAILER1オペランドを参照してください。
AVG	TRAILER1オペランドを参照してください。
SUBTOTAL	TRAILER1オペランドを参照してください。
SUBMIN	TRAILER1オペランドを参照してください。
SUBMAX	TRAILER1オペランドを参照してください。
SUBAVG	TRAILER1オペランドを参照してください。

- NODETAIL

- OUTFILでレポートを出力する際、データレコードは出力しないように設定します。つまり、レポートの内容のみ出力データセットに保存されます。

- REMOVECC

- OUTFIL出力レコードからANSI制御文字を削除して出力するように設定します。

● IFTRAIL

- 入力データセットでTRAILER1のレコードが検出されると、既存のTRAILER1のカウントおよび合計の値を更新します。
- TRAILER1レコードが検出されると、後続のレコードはすべて無視され、その前のレコードまでを計算したカウントおよび合計の値で更新されます。それにより、TRAILER1レコードの後続のレコードは、OUTFILに明示したOUTPUTデータセットにも現れません。
- IFTRAILが存在する同じOUTFILステートメント内にINCLUDE/OMITが存在する場合は、ヘッダーとTRAILER1として処理されるレコードはフィルターされません。
- 以下は、オプションについての説明です。

オプション	説明
HD=YES	入力データセットの最初のレコードをヘッダー・レコードとして処理します。 指定しない場合、最初のレコードはデータ・レコードとして扱われます。
TRLID=(logexp)	logexpを基準にしてTRAILER1レコードを検出します。 logexpと一致するレコードがなく、TRAILER1が検出されない場合、更新は行われません。
TRLUPD=(c:item,...)	itemに合計(TOT)が指定されている場合は、TRLIDのlogexpと一致するTRAILER1レコードが見つからなくても、指定されたフィールドの合計は継続して計算されるため、無効なフィールド値(数値)があると、データ例外が発生します。 - c : 出力レコードの開始位置を指定します。 - item • COUNT=(edit)または(to) • COUNT+n=(edit)または(to) • COUNT-n=(edit)または(to) • TOTAL=(p,m,f,edit)または(p,m,f,to) COUNTには、現在まで読み込んだ入力レコード数が出力され、nに指定された数値を加算または減算します。 TOTALには、指定された数値入力フィールドの合計が出力されます。TOTALの代わりにTOTを使用することができます。
p,m,f,edit	指定されたフィールドを定義された形式に変換してOUTFILレコードに出力します。 - p, m : 前述の項目説明を参照してください。

オプション	説明
	– f: フィールドの形式を指定します。 – edit: フィールドが変換される形式を定義します。EDITコマンドの詳細については、EDITコマンドを参照してください。
p,m,f,to	入力フィールドの形式を変換して出力します。 – p,m,f: 前述の項目説明を参照してください。 – to: 入力フィールドの形式fが変換される出力レコードの形式を指定します。 入力フィールドの形式fがZDFの場合は、fがZDの場合と同様に動作します。 toコマンドの構文は以下のとおりです。 <pre>of TO=[of (of)][, LENGTH=[n (n)]]</pre> • TO=[of (of)]: 変換される形式を定義します。「TO=」を省略して使用することもできます。 • LENGTH=[n (n)]: 変換される出力フィールドの長さを定義します。

- 以下は、IFTRAILの使用法と例です。

```
IFTRAIL=( [HD=YES, ]TRLID=( logexp), TRULPD=(c:item, ...))
```

```
IFTRAIL=(HD=YES, TRLID=(1,3,CH,EQ,C'TRL'), TRULPD=(11:COUNT=(M11,LENGTH=4),21:TOT=(M11,LENGTH=6)))
```

EDITコマンド

フィールドが変換される形式を定義します。EDITコマンド文は以下のとおりです。

- 使用方法

```
Mn | EDIT=(pattern)[, SIGNS=( lp, ln, tp, tn)][, LENGTH=[n | (n)]]
```

オプション	説明
pattern	I、T、S、CRと印刷可能な文字で与えられた数字フィールドが変換される編集パターンを定義します。 変換する数字フィールド(ZD、PD、BI、FIなど)に保存された値の数字と編集パターンのI、Tは後ろから1つつマッチされ変換されます。変換する数字フィールド値の絶対値の長さよりI、Tが多い場合は、超過した部分が削

オプション	説明
	除されたパターンを使用することになります。符号を示す部分は保存されます。 – I: 重要でない数字(digit)を示します。Iに該当する数字が意味のない先行0の場合、空白で代替されます – T: 重要な数字を示します。Tに該当する数字が先行0であっても0と表示されます。 – S: 符号を意味します。パターンの一番前に現れた場合、先行符号(leading sign)としてSIGNSオプションのlp、lnに置換されます。パターンの一番最後に現れた場合、終止符号(tailing sign)としてSIGNSオプションのtp、tnに置換されます。SIGNSオプションが与えられていない場合は無視されます。 – CR: 数字フィールドの値が負数の場合は右側にCRと表示され、正数の場合は右側に2つの空白が表示されます。
Mn	定義済みの編集パターンです。(0 <= n <= 26) Mnにはデフォルト値として適用されているSIGNSが存在します。
SIGNS	符号を表示する文字を定義します。 – lp: 値が正数の場合、EDIT結果の先頭に位置する符号文字を定義します。 – ln: 値が負数の場合、EDIT結果の先頭に位置する符号文字を定義します。 – tp: 値が正数の場合、EDIT結果の末尾に位置する符号文字を定義します。 – tn: 値が負数の場合、EDIT結果の末尾に位置する符号文字を定義します。
LENGTH	出力フィールドの長さを指定します。 – n: 出力フィールドの長さを指定します。(n)で使用することも可能です。

以下は、定義済みのEDIT文です。

マスク	タイプ	表示	例(値)	例(結果)
M0	IIIIIIIIIIIIIIIIIIIIITS	(none,none,blank,-)	+01234	□1234□
			-00001	□□□□1-
M1	TTTTTTTTTTTTTTTTTTTTTS	(none,none,blank,-)	-00123	00123-
			+00123	00123□
M2	II,III,III,III,III,III,III,III,II.TTTS	(none,none,blank,-)	+123450	1,234.50□
			-000020	□□□□0.20-
M3	II,III,III,III,III,III,III,III,II.TTCR	(none,none,none,none)	-001234	□□□12.34CR

マスク	タイプ	表示	例(値)	例(結果)
			+123456	1,234.56□□
M4	SII,III,III,III,III,III,III,III,IIT.TT	(+, -, none, none)	+0123456	□+1,234.56
			-1234567	-12,345.67
M5	SII,III,III,III,III,III,III,III,IIT.TTS	(blank, 左括弧, blank, 右括弧)	-001234	□□□(12.34)
			+123450	□1,234.50□
M6	III-TTT-TTTT	(none, none, none, none)	00123456	□□□□012-3456
			12345678	□□1-234-5678
M7	TTT-TT-TTTT	(none, none, none, none)	00123456	000-12-3456
			12345678	012-34-5678
M8	IT:TT:TT	(none, none, none, none)	030553	□3:05:53
			121736	12:17:36
M9	IT/TT/TT	(none, none, none, none)	123004	12/30/04
			083104	□8/31/04
M10	IIIIIIIIIIIIIIIIIIIIIT	(none, none, none, none)	01234	□1234
			00000	□□□□0
M11	TTTTTTTTTTTTTTTTTTTTTT	(none, none, none, none)	00010	00010
			01234	01234
M12	SI,III,III,III,III,III,III,III,IIT	(blank, -, none, none)	+1234567	□1,234,567
			-0012345	□□□-12,345
M13	SI.III.III.III.III.III.III.III.IIT	(blank, -, none, none)	+1234567	□1.234.567
			-0012345	□□□-12.345
M14	SI III III III III III III III IITS	(blank, 左括弧, blank, 右括弧)	+1234567	□1 234 567□
			-0012345	□□□(12 345)
M15	I III III III III III III III IITS	(none, none, blank, -)	+1234567	1 234 567□
			-0012345	□□□12 345-
M16	SI III III III III III III III IIT	(blank, -, none, none)	+1234567	□1 234 567
			-0012345	□□□-12 345
M17	SI'III'III'III'III'III'III'III'IIT	(blank, -, none, none)	+1234567	□1'234'567
			-0012345	□□□-12'345
M18	SII,III,III,III,III,III,III,III,IIT.TT	(blank, -, none, none)	+0123456	□□1,234.56

マスク	タイプ	表示	例(値)	例(結果)
			-1234567	-12,345.67
M19	SII.III.III.III.III.III.III.III.IIT,TT	(blank,-,none,none)	+0123456	□□1.234,56
			-1234567	-12.345,67
M20	SI III III III III III III III IIT,TTS	(blank,左括弧,blank,右括弧)	+0123456	□□1 234,56□
			-1234567	(12 345,67)
M21	II III III III III III III III IIT,TTS	(none,none,blank,-)	+0123456	□1 234,56□
			-1234567	12 345,67-
M22	SI III III III III III III III IIT,TT	((blank,-,none,none)	+0123456	□□1 234,56
			-1234567	-12 345,67
M23	SII'III'III'III'III'III'III'III'IIT.TT	((blank,-,none,none)	+0123456	□□1'234.56
			-1234567	-12'345.67
M24	SII'III'III'III'III'III'III'III'IIT,TT	((blank,-,none,none)	+0123456	□□1'234.56
			-1234567	-12'345.67
M25	SIIIIIIIIIIIIIIIIIIIIIIIIIT	((blank,-,none,none)	+01234	□□1234
			-00001	□□□□-1
M26	SIIIIIIIIIIIIIIIIIIIIIIIIIT	(+,-,none,none)	1234	+1234
			-1	-1

2.13. OUTREC

ソート、マージ、コピーを実行した後、その結果をSORTOUTやOUTFILで指定したデータセットに出力する前に出力レコードを再フォーマットします。INRECはソート処理を行う前にレコードを再フォーマットしますが、OUTRECはソート処理の後にレコードを再フォーマットします。

構文

```
OUTREC PARSE=(parse_item[,parse_item[,...]]) |
        FIELDS=(reformat_command[,reformat_command[,...]]) |
        BUILD=(reformat_command[,reformat_command[,...]])
        OVERLAY=(reformat_command[,reformat_command[,...]]) |
        FINDREP={IN=incon,OUT=outcon | IN=(incon1,incon2[,...]),OUT=outcon |
                INOUT=(incon1,outcon1[,incon2,outcon2[,...]])}
                [,STARTPOS=p | ENDPOS=q]
        IFTHEN=WHEN=INIT,{BUILD | OVERLAY | FINDREP} |
```



```

        WHEN=GROUP,BEGIN=( logexp),END=( logexp),RECORDS=n,PUSH=(items) |
        WHEN=(logexp),{BUILD | OVERLAY | FINDREP} |
        WHEN=NONE,{BUILD | OVERLAY | FINDREP} |
CONVERT

```

[parse_item]

```

parse_item=%nn= { FIXLEN=m | ABSPOS=p | ADDPOS=x | SUBPOS=y |
        STARTAFT=string | STARTAFT=BLANKS |
        STARTAT=string | STARTAT=BLANKS | STARTAT=NONBLANK |
        ENDBEFR=string | ENDBEFR=BLANKS |
        ENDAT=string | ENDAT=BLANKS |
        PAIR=APOST | PAIR=QUOTE }

```

[reformat_command]

```

reformat_command=[c:] | s | %nn | p,m | p,m,TRAN=LTOU|UTOL | p,m,TRAN=ALTSEQ |
        p,m,Y2x | p,m,f,edit | p,m,f,to | arexp,edit | p,m,CHANGE | p,m,a |
        DATEn | DATE | &DATE | DATE=(abcd) | &DATE=(abcd) | TIME | &TIME |
        p,m,f,dateop | p,m,Yxx,todate | TIME=(abc) | &TIME=(abc) | seqnum |
p,m,justify | %nn,justify | p,m,squeeze | %nn,squeeze

```

オペランド

以下は、オペランドについての説明です。

● PARSE

- 可変長入力レコードから解析済みの固定長フィールドを抽出します。解析済みフィールドは、BUILDまたはOVERLAYの(p,m)を使用できる場所で使用できます。
- 以下は、オプションについての説明です。

オプション	説明
%nn	可変長入力レコードを固定長出力レコードに変換します。 解析済みフィールドは、BUILDまたはOVERLAYの(p,m)を使用できる場所で使用できます。
FIXLEN=m	解析済みフィールドの長さを指定します。この属性は、すべての解析済みフィールドに存在する必要があります。
ABSPOS=p	入力レコードで解析開始位置のpを指定します。
ADDPOS=x	入力レコードで解析開始位置がxだけ増加します。
SUBPOS=y	入力レコードで解析開始位置がyだけ減少します。
STARTAFT	- STARTAFT=string : 入力レコードの最初の文字列の後から解析済みフィールドのレコードを開始します。

オプション	説明
	– STARTAFT=BLANKS : 入力レコードの最初のブランク・グループの後から解析済みフィールドのレコードを開始します。
STARTAT	– STARTAT=string : 入力レコードの最初の文字列から解析済みフィールドのレコードを開始します。 – STARTAT=BLANKS : 入力レコードの最初のブランクから解析済みフィールドのレコードを開始します。 – STARTAT=NONBLANK : 入力レコードの最初の非ブランクから解析済みフィールドのレコードを開始します。
ENDBEFR	– ENDBEFR=string : 入力レコードの最初の文字列の前のバイトで解析済みフィールドのレコードを終了します。 – ENDBEFR=BLANKS : 入力レコードの最初のブランクの前のバイトで解析済みフィールドのレコードを終了します。
ENDAT	– ENDAT=string : 入力レコードの最初の文字列の最後のバイトで解析済みフィールドのレコードを終了します。 – ENDBEFR=BLANKS : 入力レコードの最初のブランク・グループの終わりで解析済みフィールドのレコードを終了します。
PAIR	– PAIR=APOST : アポストロフィ(')で囲まれた文字列は、文字列またはブランクとして扱われません。 – PAIR=QUOTE : 引用符(")で囲まれた文字列は、文字列またはブランクとして扱われません。

● FIELDS | BUILD

- 入力レコードで選択するキー・フィールドを指定します。キー・フィールドの開始位置と長さを指定できます。

フィールドが複数指定されている場合、後ろに指定されたフィールドは、すぐ前のフィールドの後ろに付いて出力レコードとして再フォーマットされます。可変長レコード(VB)の場合、再フォーマット結果レコードの長さが変更されると、それに従ってレコード記述語(RDW)の内容も変更されます。

- FIELDSの代わりにBUILDを使用することができます。
- 以下は、オプションについての説明です。

オプション	説明
c:	c: オプションの直後に位置するフィールドの再フォーマットされた入力レコードでの開始位置を指定します。FBの場合はレコードの先頭を、VBの場合はRDWの先頭をそれぞれ1とします。

オプション	説明
	c:で指定する位置は、以前の選択フィールドや挿入された空白と異なる必要があります。このオプションによって定義されていない区間が生じた場合、その区間は空白で埋められます。
s	分離レコードとして文字列や空白を指定します。 – nX : n個の空白文字を挿入します。 – nC'xx...x' : 文字列を挿入します。nは文字列の繰り返し回数です。 – nX'yy...yy' : 16進数の文字列を挿入します。nは文字列の繰り返し回数です。
DATEn	現在の日付を出力します。OpenFrameではDATE1のみサポートしており、「C'yyyymmdd'」形式で表示されます。
TIME n	現在の時間を出力します。 nは1~3まで指定でき、以下の形式で表示されます。 – 1 : C'hhmmss' – 2 : C'hhmm' – 3 : C'hh'
DATE=(abcd) または &DATE=(abcd)	現在の日付を「C'adbdc'」形式で出力します。dは区切り子です。 a,b,c∈{M,D,Y,4} であり、それぞれの意味は以下のとおりです。 – M : 月(01~12)を2桁の数字で表示します。 – D : 日(01~31)を2桁の数字で表示します。 – Y : 年の下2桁を表示します。 – 4 : 年を4桁の数字で表示します。
DATE または &DATE	現在の日付を「C'mm/dd/yy'」形式で出力します。 – mm : 月(01-12) – dd : 日(01-31) – yy : 年の下2桁
DATENS=(abc) または &DATENS=(abc)	現在の日付を「C'abc'」形式で出力します。 a,b,c∈{M,D,Y,4} であり、それぞれの意味は以下のとおりです。 – M : 月(01~12)を2桁の数字で表示します。 – D : 日(01~31)を2桁の数字で表示します。 – Y : 年の下2桁を表示します。

オプション	説明
	– 4 : 年を4桁の数字で表示します。
TIME または &TIME	現在の時間を「C'hh:mm:ss'」形式で出力します。 – hh : 時間(00-23) – mm : 分(00-59) – ss : 秒(00-59)
TIME=(abc) または &TIME=(abc)	時間を24時間制で表示するか、あるいは12時間制で表示するかを指定します。 – ab : 24または12を指定できます。24は24時間制、12は12時間制を示します。 – c : 区切り子であり、「C'hhcmmcss'」形式で挿入されます。
p,m	指定された入力レコードのフィールドを出力フィールドに出力します。 – p : 制御フィールドの開始位置を指定します。詳細については、「2.12. OUTFIL」のOUTRECオペランドのp項目を参照してください。 – m : 選択フィールドの長さをバイト数で指定します。
%nn	%nnラベルの解析済みレコード・フィールドを出力フィールドに出力します。
p,m,YDDDNS=(ab)	指定された入力レコードのフィールドをYDDDNS=(ab)形式で表示します。 – p,m : p,mオプションの説明を参照してください。 – YDDDNS=(ab) : aには4またはY、bにはDを指定できます。4は4桁の年を、Yは年の最後の2桁を示します。Dは年間通算日(1~366)を示します。
p,m,TRAN=LTOU UTOL	– p, m : p,mオプションの説明を参照してください。 – TRAN=LTOU : 該当フィールドの小文字を大文字に変換して出力します。 – TRAN=UTOL : 該当フィールドの大文字を小文字に変換して出力します。
p,m,TRAN=ALTSEQ	– p, m : p,mオプションの説明を参照してください。 – TRAN=ALTSEQ : 該当フィールドのレコードをALTSEQ変換表に従って変更します。

オプション	説明
p,m,Y2x	– p : p,mオプションの説明を参照してください。 – m : 年を示す出力フィールドの長さを指定します。 – Y2x : OpenFrameではY2Cのみサポートしており、「C'yyyy'」形式で出力されます。
p,m,Y2x,DT=(abcd)	Y2x形式に変換されたレコード・フィールドをDT=(abcd)形式で出力します。 – p,m,Y2x : p,m,Y2xオプションの説明を参照してください。 – DT=(abcd) : Y2x形式の日付をC'adbdc'形式で出力します。dは、区切り子です。a、b、cについては、DATE=(abcd)オプションの説明を参照してください。
p,m,f,edit	指定のフィールドを定義された形式に変換してOUTFILレコードとして出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,f,to	入力フィールドの形式を変更して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
arexp,edit	算術式を編集形式に変換して入力レコードを再フォーマットします。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,CHANGE	入力フィールドの指定された文字列を他の文字列に置換します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,a	指定の入力フィールドを出力フィールドに出力します。アライメントが合わない場合、フィールドの前にヌル文字(0x0)を埋めて位置合わせをします。 aは、H、F、Dのいずれかを指定します。 – H : ハーフワード(2バイト) – F : フルワード(4バイト) – D : ダブルワード(8バイト)
p,m,f,dateop	指定された入力フィールドに、以下のように日付操作を行います。 – ADDDAYS : 指定された入力フィールドに指定の日数を加算します。 – ADDMONS : 指定された入力フィールドに指定の月数を加算します。 – ADDYEARS : 指定された入力フィールドに指定の年数を加算します。

オプション	説明
	– SUBDAYS: 指定された入力フィールドに指定の日数を減算します。 – SUBMONS: 指定された入力フィールドに指定の月数を減算します。 – SUBYEARS: 指定された入力フィールドに指定の年数を減算します。
p,m,Yxx,todate	指定された入力フィールドをtodateに指定したオプションで変換します。 todateに指定できる値は以下のとおりです。 – DT=(abcd): • a、b、c: 年(最後の2桁を示すY、4桁の年を示す4)をどのように表示するかと、月、日、年を表示する順序を示します。 • d: 年、月、日を区切るために使用します。 – TOJUL=[Yaa Yaa(s)]: 入力フィールドをユリウス日付形式で出力します。 • Yaa: ナリウス日付の形式を指定します。 ● Y2T: C'yysddd' C'yyddd' ● Y2W: C'dddssyy' C'dddyy' ● Y2U: X'yydddf' ● Y2X: X'dddyyf' ● Y4T: C'ccyysddd' C'ccyyddd' ● Y4W: C'dddscyy' C'ddccyy' ● Y4U: X'ccyydddf' ● Y4X: X'ddccyyf' • s: 区切り文字を指定します。 – TOGREG=[Yaa Yaa(s)]: 入力フィールドをグレゴリオ日付形式で出力します。 • Yaa: グレゴリオ日付の形式を指定します。 ● Y2T: C'yysmmsdd' C'yymmdd' ● Y2W: C'mmsddssyy' C'mmddy' ● Y2V: X'0yymmddf' ● Y2Y: X'0mmddyf'

オプション	説明
	• Y4T : C'ccyysmmsdd' C'ccyymmsdd' • Y4W : C'mmsddscyy' C'mmddccyy' • Y4V : X'0ccyymmsdd' • Y4Y : X'0mmddccyy' • s : 区切り文字を指定します。
seqnum	出力レコードのシーケンス番号を出力します。シーケンス番号は、入力レコードの後に追加されます。 以下は、OpenFrameでサポートされる形式です。 <pre>SEQNUM,n,fs[,START=j][,INCR=i][,RESTART=(p,m)]</pre> – SEQNUM : シーケンス番号を出力するコマンドを指定します。 – n : シーケンス番号の桁数を指定します。必須オプションです。 – fs : シーケンス番号の文字列形式を指定します。必須オプションです。 – START=j : シーケンス番号の開始値をjに設定します。このオプションは省略可能であり、省略した場合は、START=1と見なされます。 – INCR=i : シーケンス番号の増分値をiに設定します。このオプションは省略可能であり、省略した場合は、INCR=1と見なされます。 – RESTART=(p,m) : このオプションは省略できます。指定されたフィールドが前のレコードのフィールドと一致しない場合、SEQNUMはjに初期化されます。
p,m,justify	指定されたフィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
%nn,justify	%nnラベルの解析済みレコード・フィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
p,m,squeeze	指定されたフィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。
%nn,squeeze	%nnラベルの解析済みレコード・フィールドを定義された形式に変換して出力します。詳細については、 「2.12. OUTFIL」 のOUTRECオペランドを参照してください。

- OVERLAY
 - 出力レコードに上書きする項目を指定します。上書きされていない列はそのまま出力します。
- FINDREP
 - 入力レコードで指定された変数を検索して他の変数に置換します。
 - 変数の置換についての詳細は、[「2.12. OUTFIL」](#)のFINDREPオペランドを参照してください。
- IFTHEN
 - 与えられた条件に従って、各レコードを異なる方式で再フォーマットできます。
 - 条件設定についての詳細は、[「2.12. OUTFIL」](#)のIFTHENオペランドを参照してください。
- CONVERT
 - 可変長レコードを固定長レコードに変換します。

2.14. RECORD

レコードの形式および長さを指定します。

構文

```
RECORD [TYPE=[F|V|D] [,LENGTH=[(L1)|(L1,L2,L3,L4,L5)]]
```

オペランド

以下は、オペランドについての説明です。

- TYPE
 - レコードの形式を指定します。
 - 以下は、設定項目についての説明です。

項目	説明
F	固定長レコードを意味します。FBとも記述できます。
V	可変長レコードを意味します。VBとも記述できます。
D	ASCII可変長レコードを意味します。DBとも記述できます。OpenFrameでは構文解析のみサポートします。

- 入力データセットがVSAMの場合、TYPEを使用してFBまたはVBで処理できます。この際、Fを指定すると入力レコードをそのまま処理し、Vを指定した場合は、入力レコードの前にRDWを

加えて処理します。入力データセットが非VSAMの場合、入力データセットのRECFM情報を確認してから形式を決めます。TYPEは無視します。

- LENGTH

- レコードの長さを指定します。
- 固定長レコードの場合は、L1からL3まで、可変長レコードの場合はL1からL5までを指定できます。L1を除いてすべて省略が可能です。
- 以下は、設定項目についての説明です。

項目	説明
L1	必須入力項目であり、入力レコードの長さを指定します。 FBの場合は入力レコードの長さを指定し、VBの場合は入力レコードの最大長を指定します。 SORTINのLRECL(logical record length)が与えられなかった場合に使用できます。レコード制御文が存在しない場合は、DCBパラメータ内に定義された値に従います。
L2	FBの場合はソート処理中のレコード長を、VBの場合はソート処理中のレコードの最大長を指定します。省略した場合は、L1に指定した値が設定されます。 OpenFrameでは構文解析のみサポートします。
L3	FBの場合は出力レコードの長さを指定し、VBの場合は出力レコードの最大長を指定します。INRECやOUTRECがある場合は、処理後の長さを示し、INREC/OUTRECがない場合はL1の値が指定されます。
L4	入力レコードの最小の長さを指定します。 OpenFrameでは構文解析のみサポートします。
L5	入力レコードの平均の長さを指定します。省略した場合は、L1とL4の平均が設定されます。ただし、マージの場合は省略されても他の値が指定できません。 OpenFrameでは構文解析のみサポートします。

例

以下は、レコード長が200バイトの固定長レコードを指定する例です。

```
RECORD TYPE=F, LENGTH=(200)
```

以下は、最大長が100バイト、最小長が50バイトの可変長レコードを指定する例です。

```
RECORD TYPE=V, LENGTH=(100,,,50)
```

2.15. REFORMAT

OpenFrameでは結合を実行する際にREFORMAT文を指定することを推奨します。結合を実行時にはレコードの指定した部分のみを出力します。REFORMAT文を指定しない場合は、JOIN文に従ってF1またはF2に指定されたデータセットの最大長でREFORMAT文を指定します。

構文

```
REFORMAT FIELDS=( {F1|F2}:p,m[,p,m[,...]][,{F1|F2}:p,m[,p,m[,...]][,...]] )
```

オペランド

以下は、オペランドについての説明です。

• FIELDS

- 入力レコードで選択するキー・フィールドを指定します。キー・フィールドの開始位置と長さを指定することができます。

オプション	説明
F1	F1を指定すると、左のデータセット(SORTJNF1)のレコードでキー・フィールドを選択します。
F2	F2を指定すると、右のデータセット(SORTJNF2)のレコードでキー・フィールドを選択します。

複数のフィールドを指定する場合、後に指定されるフィールドはすぐ前に指定されたフィールドの後ろに付いて出力レコードとして再フォーマットされます。

入力データセットのRECFMのどちらかがVタイプの場合は、フィールドにRDWが含まれる必要があります。

- 以下は、オプションについての説明です。

オプション	説明
p,m	指定した入力レコードのフィールドを出力フィールドに出力します。 – p: 制御フィールドの開始位置を指定します。詳細については、「2.12. OUTFIL」のOUTRECオペランドのp項目を参照してください。 – m: 選択フィールドの長さをバイト数で指定します。

- 以下は、SORTJNF1とSORTJNF2を結合する例です。SORTJNF1のレコードの最初の17バイトとSORTJNF2のレコードの最初の10バイトのみを出力します。

```
JOINKEYS FILE=F1,FIELDS=(11,7,A)
JOINKEYS FILE=F2,FIELDS=(11,7,A)
REFORMAT FIELDS=(F1:1,17,F2:1,10)
```

- 以下は、REFORMAT文を指定していない場合にSORTJNF1とSORTJNF2を結合する例です。SORTJNF1とSORTJNF2の最大長は7です。

OpenFrameでは以下のREFORMAT文を追加します。

```
JOINKEYS FILE=F1,FIELDS=(1,7,A)
JOINKEYS FILE=F2,FIELDS=(1,7,A)
JOIN UNPAIRED,ONLY,F1,F2
```

```
REFORMAT FIELDS=(F1:1,7,F2:1,7)
```

2.16. SORT

入力データセットのレコードを指定されたキーに従って昇順または降順にソートし、出力データセットに出力します。

OUTFILのFTOV、VTOFオプションを使用しない場合は、入力データセットのRECFMと出力データセットのRECFMが一致する必要があります。

構文

```
SORT FIELDS=(p,m,f,s[,p,m,f,s[,...]]) | (p,m,s[,p,m,s[,...]]),FORMAT=f | COPY
[SKIPREC | STOPAFT | CKPT | DYNALLOC | EQUALS | NOEQUALS | FILSZ=[E|U]n |
SIZE=[E|U]n | Y2PAST | CENTWIN]
```

オペランド

以下は、オペランドについての説明です。

• FIELDS

- 入力レコードをソートするためのキー・フィールドを指定します。キー・フィールドの開始位置、長さ、形式、ソートの基準を決めることができます。(別名: KEYS)
- 以下は、オプションについての説明です。

オプション	説明
p	制御フィールドの開始位置を指定します。制御フィールドは、場合に応じて開始位置が異なります。

オプション	説明
	– 入力レコードが非VSAM固定長(FB)の場合、入力レコードの1番目のバイトが1となります。 – 入力レコードが非VSAM可変長(VB)の場合、レコードの前に4バイトのレコード記述語(RDW)が追加されるので、RDWの開始点が1となり、実際のレコードは5から始まります。 – 入力レコードがVSAMデータセットであり、OpenFrame SORTでこれを可変長レコードとして処理する場合、RDWをレコードの前に追加し、その開始点を1とします。したがって、結合を実行する際に実際のデータは5から始まります。 また、OpenFrame SORTの実行時に、VSAMデータセットを固定長レコードと判断するか、あるいは可変長レコードと判断するかを決める必要があります。以下のいずれかの条件を満たす場合、そのデータセットは可変長レコードと判断されます。 1. RECORD TYPE=Vと指定されている場合 2. SORTOUTがRECFM=VBと指定されている場合 3. SORTINとSORTOUTが両方ともVSAMの場合
m	制御フィールドの長さを指定します。
f	制御フィールドの形式を指定します。指定可能な制御フィールドの形式と形式に伴う長さについての説明は以下の表を参照してください。
s	制御フィールドのソート基準を指定します。 – A(Ascending order) : 昇順 – D(Descending order) : 降順 – E : ユーザー定義の順序 (OpenFrameではサポートされません)

以下の表は、指定可能な制御フィールドの形式と長さをまとめたものです。

形式	説明	長さ	符号
CH	文字(ASCII)	1～32760バイト	なし
AC	文字(ASCII)	1～32760バイト	なし
ZD (Zoned Decimal)	ゾーン10進数	1～32760バイト	あり
PD (Packed Decimal)	パック10進数	1～32760バイト	あり
PDF	正の数の符号ビットがFのパック10進数	1～32760バイト	あり

形式	説明	長さ	符号
PD0	最上位ニブルと最下位ニブルを除いたパック10進数	1～32760バイト	なし
FI (Fixed-point)	固定小数点の2進数	1～32760バイト	あり
FL (Floating-point)	浮動小数点の10進数	1～32760バイト	あり
BI (Binary Integer)	2進数	1～32760バイト	なし
CSF or FS	追加の前置浮動符号付数字	1～32760バイト	あり
UFF	符号がない自由形式の数字	1～32760バイト	なし
SFF	符号がある自由形式の数字	1～32760バイト	あり
CSL	前置分離符号付数字	1～32760バイト	あり
CST	後置分離符号付数字	1～32760バイト	あり
CLO	前置オーバパンチ符号付数字	1～32760バイト	あり
CTO	後置オーバパンチ符号付数字	1～32760バイト	あり
ASL	前置分離符号付数字、ASCII	1～32760バイト	あり
AST	後置分離符号付数字、ASCII	1～32760バイト	あり
AQ	文字(ALTSEQで定義した順序)	1～32760バイト	なし
Y2C or Y2Z	2桁の文字またはZDの年データ	2バイト	なし
Y2P	2桁のPDの年データ	2バイト	なし
Y2D	2桁の10進の年データ	1バイト	なし
Y2S	2桁の文字またはZDの年データ	2バイト	なし
Y2B	2桁の2進の年データ	1バイト	なし

- 以下は、形式が文字である固定長レコードの1番目のバイトから始まり、7バイトまでの内容を昇順にソートする例です。

```
SORT FIELDS=(1,7,CH,A)
```

以下は、可変長レコードの場合、1から4までのRDWを除き、実際のレコードの先頭の5から3バイト分の文字形式のレコードを降順にソートする例です。

```
SORT FIELDS=(5,3,CH,D)
```

参考

可変長レコードの場合、RDWを除いたレコードの先頭は5です。そのため、ソートの開始点を1から4までの間に指定した場合、エラーは発生しないものの、RDWが含まれるのでソートが正常に実行されません。

- FORMAT=f

- 制御フィールドの形式がすべて一致する場合、制御フィールド内で形式を指定せず、外部からFORMAT=fコマンドを使用して一括指定できます。
- 以下は、FORMATの例です。2つのCH形式の固定長制御フィールドを指定します。レコードの最初の1バイトから10バイト分を昇順にソートし、11バイトから10バイト分を降順にソートします。

```
SORT FIELDS=(1,10,A,11,10,D),FORMAT=CH
```

参考

OpenFrameでFORMAT=fオペランドを指定するときは、制御フィールドの形式がすべて同じである必要があります。たとえば、複数の制御フィールドのうち1つのフィールド形式が異なり、残りはすべて同じである場合、形式が異なるフィールドは内部で指定し、それ以外のフィールドの形式は外部からFORMAT=fで指定するとエラーが発生します。FORMAT=fをフィールドの後ろではなく、前に記述する場合にもエラーが発生するので注意してください。

- COPY

- COPYコマンドは、入力データセット・レコードの内容を制御フィールドの指定なしで出力データセットにそのままコピーします。
- 以下は、COPYの例です。

```
SORT FIELDS=COPY
```

- FILSZ=[E|U]n

- 実際にソートに使われるレコードの数を指定します。
- OpenFrameでは構文解析のみサポートします。

- SIZE=[E|U]n

- 入力レコードの数を指定します。
- 詳細については、[「2.11. OPTION」](#)を参照してください。

- SKIPREC=x

- 入力レコードの1番目のレコードから、ソートせずに削除(discard)するレコードの数を指定します。
- xはレコードの数を意味します。
- 以下は、SKIPRECの例です。入力データセットの1番目のレコードから5番目のレコードまでを除いたレコードを、制御フィールドに記述した内容に従ってソートします。

```
SORT FIELDS=(1,7,CH,A),SKIPREC=5
```

- STOPAFT=x

- 入力レコードの1番目のレコードから、ソートするレコードの数を指定します。指定された数を越えたレコードは削除されます。
- xはレコードの数を意味します。
- 以下は、STOPAFTの例です。入力データセットの1番目のレコードから5番目のレコードまでを、制御フィールドに記述した内容に従ってソートし、残りのレコードは削除します。

```
SORT FIELDS=(1,7,CH,A),STOPAFT=5
```

- CKPT

- ソートまたはコピーを使用する際のチェックポイントを指定します。
- OpenFrameでは構文解析のみサポートします。

- DYNALLOC [=([d],[n])]

- 処理用データセットを動的に割り当てます。
- 以下は、設定項目についての説明です。

項目	説明
d	動的に割り当てるデバイスのタイプまたはグループ名を指定します。
n	動的に割り当てる処理用データセットの数を指定します。

- OpenFrameでは構文解析のみサポートします。

- EQUALS | NOEQUALS

- 制御フィールドの内容がすべて同じレコードが入力されている場合の出力順を指定します。
- 以下は、設定項目についての説明です。

項目	説明
EQUALS	入力順に出力されます。(安定なソート、別名: SEQ)
NOEQUALS	入力順に出力されることを保証しません。(不安定なソート、デフォルト値)

- Y2PAST | CENTWIN

- 詳細については、[「2.11. OPTION」](#)を参照してください。

2.17. SUM

ソート、マージを実行する際、同じキー値を持つレコードに対して与えられたフィールドの値をすべて合計し、1つのレコードとして表示します。合計後の1つのレコードを除き、残りのレコードは

すべて削除されます。また、キー値が同じレコードのフィールドを合計せずに、1つのレコードだけ残し、その他のレコードはすべて削除する機能も提供します。

構文

```
SUM FIELDS=(p,m,f,[p,m,f[,...]])|(p,m[,p,m[,...]],FORMAT=f|NONE[,XSUM]
```

オペランド

以下は、オペランドについての説明です。

● FIELDS

- 入力レコードを合計するためのキー・フィールドを指定します。
- キー・フィールドの開始位置と長さ、形式を指定します。
- 以下は、設定項目についての説明です。

項目	説明
p	合計を実行するフィールドの開始位置を指定します。詳細については、 「2.16. SORT」 のFIELDSオペランドのp項目を参照します。
m	合計するフィールドの長さをバイト単位で指定します。
f	合計フィールドの形式を指定します。
s	制御フィールドのソート順を指定します。 – A (Ascending order) : 昇順 – D (Descending order) : 降順 – E : ユーザー定義の順序 (OpenFrameではサポートされません)

- 以下は、ZD形式のレコードに対して開始位置10から10バイトを合計する例です。

```
SUM FIELDS=(10,10,ZD)
```

● FORMAT=f

- 合計フィールドの形式がすべて同じである場合は、各フィールド内で形式を指定せずに、CONDオペランドの後ろにFORMAT=fを使用して一度に指定することができます。
- 以下は、10から2バイト、12から4バイト、16から8バイトのFI形式のレコードを合計する例です。

```
SUM FIELDS=(10,2,12,4,16,8),FORMAT=FI
```

参考

OpenFrameでは制御フィールドの形式がすべて同じである場合にのみFORMATオペランドを指定することができます。

- NONE

- レコードを合計せずに、同じキー値を持つレコードを1つのみ残し、それ以外はすべて削除する場合に使用します。
- 以下は、同じキー値を持つレコードを1つのみ残し、それ以外はすべて削除する例です。

```
SUM FIELDS=NONE
```

- XSUM

- 同じキー値を持つレコードのうち削除されるレコードを、DD名がSORTXSUMのデータセットに保存する場合に使用します。
- 以下は、同じキー値を持つレコードのうち削除されるレコードを、DD名がSORTXSUMのデータセットに保存する例です。

```
SUM FIELDS=NONE, XSUM
```

注意事項

以下は、SUM文を作成する際の注意事項です。

- ソート、マージでNOEQUALSが指定されている場合は、合算および削除されるレコードはランダムに決定されます。
 - ソートで、EQUALSが指定されている場合は、先に入力されたレコードに残りのレコードを合算します。
 - マージで、EQUALSが指定されている場合は、SORTINnn番号が一番小さいレコードに合算されます。SORTINnn番号が同じである場合は、先に入力されたレコードに合算されます。
- SUMの実行時に、指定したレコード・フィールドの内容のみ合算され、他のフィールドの内容は合算されません。
- SUMの実行時に、オーバーフローが発生した場合は合算されません。また、合算されなかったレコードは削除されません。

索引

A

ALTSEQコマンド, 10
ALTSEQコマンドのオペランド
CODE, 10

D

DUPKEYSコマンド, 11
DUPKEYSコマンドのオペランド
ALLDUPS, 11
AVG|MAX|MIN|SUM, 12
FIELDS=NONE, 11
FIRSTDUP[
NODUPS], 12
FORMAT, 11
LASTDUP[
NODUPS], 12
NODUPS, 12
XDUP, 12

E

ENDコマンド, 14

I

INCLUDE/OMITコマンド, 14
INCLUDE/OMITコマンドのオペランド
COND, 14
FORMAT, 15
relational_conditionオペランド, 15
INRECコマンド, 17
INRECコマンドのオペランド
FIELDS | BUILD, 19
FINDREP, 24
IFTHEN, 24
OVERLAY, 24
PARSE, 18

J

JOINKEYSコマンド, 25
JOINコマンド, 24

M

MERGEコマンド, 28
MERGEコマンドのオペランド
CKPT, 31
COPY, 30
EQUALS \| NOEQUALS, 31
FIELDS, 28
FILES, 31
FILSZ, 31
FORMAT, 29
SIZE, 31
SKIPREC, 30
STOPAFT, 30
Y2PAST \| CENTWIN, 31
MODSコマンド, 31
MODSコマンドのオペランド
exit, 32

O

OPTIONコマンド, 34
OPTIONコマンドのオプション
CENTWIN, 37
CKPT, 35
COPY, 34
DYNALLOC, 35
FILSZ, 35
MSGPRT, 35
NULLOUT, 37
OVFLO, 35
PRINT, 35
SIZE, 35
SOLRF| NOSOLRF, 36
SORTDD, 36
SORTIN, 36
SORTOUT, 36
VLSHRT|NOVLSHRT, 37
VLTESTI, 38
Y2PAST, 36

ZDPRINT | NZDPRINT, 36
OUTFILコマンド, 38
OUTFILコマンドのオペランド
 ENDREC, 39
 FILES, 39
 FINDREP, 47
 FNAMES, 39
 FTOV, 49
 HEADER1, 50
 HEADER2, 52
 IFTHEN, 48
 IFTRAIL, 56
 INCLUDE | OMIT, 39
 LINES, 50
 NODETAIL, 55
 NULLOFL, 49
 OUTREC | BUILD, 41
 OVERLAY, 47
 PARSE, 40
 REMOVECC, 55
 SAVE, 40
 SECTIONS, 53
 SPLIT | SPLITBY | SPLIT1R, 49
 STARTREC, 39
 TRAILER1, 51
 TRAILER2, 52
 VLFILL, 49
 VLTRIM, 49
 VTOF | CONVERT, 49

OUTRECコマンド, 60

OUTRECコマンドのオペランド

 CONVERT, 68
 FIELDS | BUILD, 62
 FINDREP, 68
 IFTHEN, 68
 OVERLAY, 68
 PARSE, 61

R

RECORDコマンド, 68

RECORDコマンドのオペランド

 LENGTH, 69

 TYPE, 68
REFORMATコマンド, 70
REFORMATコマンドのオペランド
 FIELDS, 70

S

SORTコマンド, 71

 ALTSEQ, 10
 DUPKEYS, 11
 END, 14
 INCLUDE/OMIT, 14
 INREC, 17
 JOIN, 24
 JOINKEYS, 25
 MERGE, 28
 MODS, 31
 OPTION, 34
 OUTFIL, 38
 OUTREC, 60
 RECORD, 68
 REFORMAT, 70
 SORT, 71
 SUM, 75

SORTコマンドのオペランド

 CKPT, 75
 COPY, 74
 DYNALLOC, 75
 EQUALS | NOEQUALS, 75
 FIELDS, 71
 FILSZ, 74
 FORMAT, 74
 SIZE, 74
 SKIPREC, 74
 STOPAFT, 75
 Y2PAST | CENTWIN, 75

SUMコマンド, 75

SUMコマンドのオペランド

 FIELDS, 76
 FORMAT, 76
 NONE, 77
 XSUM, 77

さ

ジョブ制御文, 2

/*文, 4

DD文, 3

EXEC文, 2

JOB文, 2

は

ブロック・サイズ(BLKSIZE), 5

ら

レコード形式(RECFM), 5

レコード長(LRECL), 5

